Hệ thống nhúng

Tuần 3 — Lập trình C cho hệ thống nhúng

TS. Nguyễn Mạnh Hùng

Chương trình 9 tuần — Buổi 3

Học phần: Hệ thống nhúng (Embedded Systems)

Mục tiêu buổi học

Chuẩn đầu ra liên quan

CLO2: Lập trình được bằng hợp ngữ/C cho vi điều khiển.

Sau tuần 3, sinh viên có khả năng:

- Giải thích vì sao ngôn ngữ C được dùng phổ biến cho lập trình hệ thống nhúng, và luồng biên dịch avr-gcc.
- Sử dụng đúng các kiểu dữ liệu chuẩn C và kiểu chính xác cố định (`stdint.h`) trên vi điều khiển AVR 8-bit.
- Giải thích vai trò của `volatile`, `const`/`PROGMEM`, toán tử bit-wise và macro tiền xử lý trong lập trình nhúng.
- Phân tích chi phí tính toán số học (cộng/trừ/nhân/chia, số thực) trên vi điều khiển 8-bit và biết khi nào nên tránh.
- Viết chương trình C điều khiển còi Piezo và giao tiếp UART cơ bản trên Arduino Uno.

Nội dung buổi học

Vì sao dùng C cho hệ thống nhúng

Kiểu dữ liệu và biến trong C nhúng

Số học và giới hạn của C trên vi điều khiển 8-bit

Tổng kết chương

Thực hành — Tuần 3

Vì sao dùng C cho ARM Cortex-M

Kiểu dữ liệu và biến trong C trên Cortex-M

Số học và đặc điểm của C trên Cortex-M 32-bit

Thực hành — ARM Cortex-M: Buzzer & UART

Tổng kết & So sánh: C trên AVR và C trên Cortex-M

Vì sao dùng C cho hệ thống nhúng

Nhắc lại: Lập trình hợp ngữ AVR (Tuần 2)

- Tuần 2 đã lập trình trực tiếp bằng hợp ngữ (Assembly): từng lệnh `LDI`, `OUT`, `SBI`, `RJMP`... ánh xạ đúng 1:1 với một lệnh máy của CPU AVR.
- Ưu điểm: kiểm soát tuyệt đối phần cứng, hiệu năng tối đa, kích thước mã tối thiểu.
- Nhược điểm cảm nhận được: chương trình dài, khó đọc lại, mất nhiều thời gian cho các phép toán tưởng chừng đơn giản (như cộng 2 số 16-bit).
- Tuần 3: chuyển sang ngôn ngữ **C** — vẫn cùng một phần cứng AVR, nhưng năng suất lập trình cao hơn nhiều.

Vì sao không lập trình thuần Assembly cho mọi hệ nhúng?

- Chương trình thực tế có hàng nghìn–hàng chục nghìn dòng lệnh — viết và bảo trì bằng Assembly rất tốn thời gian, dễ lỗi.
- Assembly gắn chặt với kiến trúc CPU cụ thể: đổi sang vi điều khiển khác gần như phải viết lại toàn bộ.
- Khó làm việc nhóm: mã Assembly của người khác thường khó đọc, khó tái sử dụng.
- Áp lực thời gian ra thị trường (time-to-market) đòi hỏi ngôn ngữ năng suất cao hơn nhưng vẫn đủ gần phần cứng.

Ngôn ngữ C: ”Hợp ngữ bậc cao” cho hệ nhúng

Vị trí của C

C là ngôn ngữ lập trình bậc trung (mid-level): đủ trừu tượng để viết biểu thức, hàm, cấu trúc điều khiển như một ngôn ngữ bậc cao, nhưng vẫn đủ gần phần cứng để thao tác trực tiếp thanh ghi, con trỏ, bộ nhớ như Assembly.

- Không có máy ảo (virtual machine), không thu gom rác (garbage collector) tự động — phù hợp hệ thống tài nguyên hạn chế.
- Hơn 95% firmware nhúng trên thế giới hiện nay được viết bằng C hoặc C/C++.

C vs Assembly: Năng suất lập trình (Productivity)

- C cho phép biểu diễn phép toán phức tạp bằng biểu thức: `c = (a + b) * 2;` thay vì nhiều dòng `ADD`/`MUL`.
- Hàm (function), thư viện chuẩn, cấu trúc điều khiển (`if`, `for`, `while`) giúp tổ chức mã rõ ràng.
- Trình biên dịch tự động xử lý các việc tẻ nhạt: cấp phát thanh ghi, tính toán đa byte, nhảy có điều kiện.
- Hệ quả: thời gian phát triển và số lỗi giảm đáng kể so với Assembly cho cùng một chức năng.

C vs Assembly: Hiệu năng và kiểm soát phần cứng

- Assembly cho phép kiểm soát **từng chu kỳ xung nhịp**, từng thanh ghi — quan trọng cho các đoạn mã cực kỳ nhạy thời gian (ISR tối giản, driver tốc độ cao).
- Trình biên dịch C hiện đại (avr-gcc) với tối ưu hoá (`-0s`, `-02`) thường sinh mã gần tối ưu như Assembly viết tay cho các đoạn mã đơn giản.
- C không loại bỏ khả năng truy cập phần cứng: vẫn thao tác trực tiếp thanh ghi qua địa chỉ bộ nhớ (`DDRB`, `PORTB`...).
- Đánh đổi: C thường sinh nhiều lệnh hơn Assembly viết tay cho các thao tác phức tạp — sẽ minh hoạ cụ thể ở các slide sau.

Bảng so sánh tổng hợp C và Assembly

Tiêu chí	Assembly	C
Năng suất lập trình	Thấp — viết từng lệnh máy	Cao — biểu thức, hàm, thư viện
Tính di động (portability)	Không — gắn chặt kiến trúc CPU	Cao — biên dịch lại cho MCU khác
Dễ đọc / bảo trì	Khó với chương trình lớn	Dễ hơn nhiều
Kiểm soát phần cứng	Tuyệt đối, từng chu kỳ	Gián tiếp, qua trình biên dịch
Hiệu năng tối ưu	Tối đa (nếu tay nghề cao)	Rất tốt với -0s/-02
Thời gian phát triển	Dài	Ngắn hơn nhiều

Khi nào vẫn cần Assembly trong dự án dùng C?

- Đoạn mã khởi động hệ thống cực sớm (startup code) — trước khi ngăn xếp (stack) sẵn sàng.
- Trình phục vụ ngắt (ISR) cực kỳ nhạy thời gian, cần tối ưu từng chu kỳ.
- Truy cập lệnh CPU đặc biệt mà C không biểu diễn trực tiếp được (một số lệnh AVR chuyên dụng).
- Thực tế phổ biến: viết **đa số bằng C**, chỉ chèn Assembly nội tuyến (*inline assembly*) cho vài đoạn cực nhỏ khi thật sự cần thiết.

Trình biên dịch avr-gcc và avr-libc

- **avr-gcc**: phiên bản GCC (GNU Compiler Collection) được biên dịch chéo (cross-compiler) để sinh mã máy cho kiến trúc AVR, chạy trên máy tính PC thông thường.
- **avr-libc**: thư viện chuẩn C dành riêng cho AVR — cung cấp `avr/io.h` (định nghĩa thanh ghi), `util/delay.h` (hàm trễ), `avr/interrupt.h` (ngắt)...
- Arduino IDE thực chất dùng avr-gcc/avr-libc bên dưới — nút *Verify*/*Upload* chỉ là giao diện thân thiện hơn cho cùng bộ công cụ này.

Bộ công cụ AVR Toolchain

- `avr-gcc` — biên dịch mã nguồn C/C++ thành mã máy AVR.
- `avr-libc` — thư viện chuẩn C cho AVR (I/O, delay, ngắt, PROGMEM...).
- `avr-as` — hợp dịch (assembler): sinh mã máy từ hợp ngữ.
- `avr-ld` — liên kết (linker): gộp các file object + thư viện thành file thực thi.
- `avr-objcopy` — chuyển định dạng file (`.elf` $\rightarrow$ `.hex`) để nạp vào vi điều khiển.
- `avr-objdump` — dịch ngược (disassemble) mã máy để kiểm tra.
- `avrdude` — nạp file `.hex` vào bộ nhớ Flash qua ISP/bootloader.

Luồng biên dịch: Preprocess → Compile → Assemble → Link



Lệnh `avr-gcc main.c -o main.elf` thực hiện tuần tự cả 4 bước này (thường trong 1 lệnh duy nhất).

Bước 1: Tiền xử lý (Preprocessing)

- Do bộ tiền xử lý C (*C preprocessor*, cpp) thực hiện, trước khi trình biên dịch thật sự đọc mã.
- Mở rộng chỉ thị `#include` — chèn toàn bộ nội dung file header vào chỗ gọi.
- Mở rộng macro `#define` — thay thế văn bản đơn thuần.
- Xử lý biên dịch có điều kiện: `#ifdef`, `#ifndef`, `#endif`.
- Kết quả trung gian: file `.i` — mã C thuần, không còn chỉ thị tiền xử lý.

Bước 2–3: Biên dịch và Hợp dịch (Compile & Assemble)

- **Biên dịch (Compile)**: `avr-gcc` phân tích cú pháp file `.i`, kiểm tra kiểu dữ liệu, tối ưu hoá, sinh ra mã hợp ngữ AVR tương ứng (file `.s`).
- **Hợp dịch (Assemble)**: `avr-as` dịch file `.s` (hợp ngữ dạng văn bản) thành mã máy nhị phân — file đối tượng (object file) `.o`.
- File `.o` chưa phải chương trình hoàn chỉnh: các lời gọi hàm ở thư viện khác (như `_delay_ms`) chỉ là tham chiếu chưa được giải quyết (unresolved reference).

Bước 4: Liên kết (Linking) và tạo file nạp

- **Liên kết (Link)**: `avr-ld` kết hợp mọi file `.o` với các hàm thư viện cần thiết từ `avr-libc`, giải quyết mọi tham chiếu còn treo.
- Kết quả: file `.elf` — chương trình hoàn chỉnh, có thông tin ánh xạ bộ nhớ và debug.
- `avr-objcopy` chuyển `.elf` sang định dạng `.hex` (Intel HEX) — định dạng mà `avrdude` nạp được vào Flash của vi điều khiển.

Lưu ý

Chỉ file `.hex` (hoặc `.bin`) mới được nạp vào vi điều khiển thật — `.elf` chỉ dùng trên máy tính để debug/phân tích.

Cấu trúc chương trình C nhúng cơ bản

```c
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    // Khoi tao (init): cau hinh thanh ghi, ngoai vi 1 lan
    DDRB |= (1 << PB5);

    while (1) {              // Vong lap vo han (super-loop)
        // Xu ly chinh: doc cam bien, dieu khien ngo ra...
        PORTB ^= (1 << PB5);
        _delay_ms(500);
    }
    // Khong bao gio den duoc day
}
```

Cấu trúc điển hình: **khởi tạo một lần** rồi **lặp vô hạn** — khác hẳn chương trình PC (chạy xong rồi thoát).

Hàm main() và vòng lặp vô hạn

- Mọi chương trình C đều bắt đầu thực thi tại hàm `main()` — quy tắc này giữ nguyên trên hệ nhúng.
- Khác biệt cốt lõi: chương trình nhúng **không bao giờ kết thúc** khi thiết bị còn cấp nguồn — nên thân hàm `main()` luôn chứa một vòng lặp vô hạn (`while(1)` hoặc `for(;;)`), thường gọi là *super-loop*.
- Mọi công việc lặp lại của hệ thống (đọc cảm biến, điều khiển ngõ ra, kiểm tra sự kiện) nằm bên trong vòng lặp này.

Vì sao main() không return trong hệ nhúng?

Không có hệ điều hành để ”quay về”

Trên PC, khi `main()` return, hệ điều hành thu hồi tiến trình. Trên vi điều khiển **không có hệ điều hành** — nếu `main()` return, không có nơi nào để ”quay về” cả.

- avr-libc xử lý an toàn: nếu `main()` lỡ return, chương trình khởi động sẽ rơi vào một vòng lặp vô hạn nội bộ (không làm gì) — không có hành vi hữu ích.
- Vì vậy hàm `main()` trong lập trình nhúng luôn được khai báo `int main(void)` nhưng thực chất không bao giờ dùng lệnh `return`.

Đọc mã Assembly sinh từ C bằng avr-objdump

- Lệnh `avr-objdump -S chương_trình.elf` in ra mã Assembly AVR xen kẽ với dòng mã C tương ứng — cách trực quan nhất để hiểu trình biên dịch ”nghĩ” gì.
- Hữu ích khi: nghi ngờ lỗi tối ưu hoá, muốn kiểm tra kích thước mã, hoặc học cách trình biên dịch dịch một cấu trúc C cụ thể.
- Định dạng mỗi dòng: *địa chỉ: mã máy (hex) lệnh Assembly chú thích*.

Ví dụ: từ C sang Assembly — mã nguồn C

```c
#include <avr/io.h>

void bat_led(void) {
    PORTB |= (1 << PB5);   // Set bit 5 cua PORTB, giu nguyen cac bit khac
}
```

Biên dịch bằng: `avr-gcc -mmcu=atmega328p -Os -c bat_led.c -o bat_led.o`

Ví dụ: từ C sang Assembly — kết quả disassembly

```
00000000 <bat_led>:
   0:   2d 9a         SBI  0x05, 5    ; PORTB |= (1<<PB5)
   2:   08 95         RET
```

- Trình biên dịch nhận ra `PORTB |= (1<<PB5)` là ”set đúng 1 bit, giữ nguyên các bit khác” $\rightarrow$ dịch thẳng thành **một lệnh duy nhất** `SBI` (Set Bit in I/O register).
- Đây chính là lệnh mà lập trình viên Assembly (Tuần 2) sẽ tự tay viết — trình biên dịch C tối ưu tốt cho mẫu hình phổ biến này.

So sánh hiệu năng: C tối ưu vs Assembly viết tay

Thao tác	Assembly tay	C (avr-gcc -0s)
Set 1 bit (`PORTB |= ...`)	1 lệnh (`SBI`)	1 lệnh (`SBI`)
Clear 1 bit (`PORTB &= ...`)	1 lệnh (`CBI`)	1 lệnh (`CBI`)
Toggle qua XOR (`PORTB ^= ...`)	1 lệnh (`IN`+`EOR`+`OUT` thủ công)	4 lệnh (`IN`/`LDI`/`EOR`/`OUT`)

- Với mẫu hình đơn giản (set/clear 1 bit), C tối ưu cho ra **đúng bằng** Assembly viết tay — không mất hiệu năng.
- Với phép XOR-toggle, cả C lẫn Assembly ngây thơ đều cần đọc-sửa-ghi 3–4 lệnh — trình biên dịch không ”thông minh hơn” phần cứng cho phép.

Ví dụ callback: cùng chức năng — Assembly (Tuần 2) vs C (Tuần 3)

- Chương trình chớp tắt LED (Lab 1, Tuần 1) viết bằng C với `DDRB`, `PORTB`, `_delay_ms()` biên dịch (`-Os`) chỉ ra **162 byte** mã máy — gồm 1 lệnh `SBI` khởi tạo, vòng lặp đọc-sửa-ghi để đảo LED, và một vòng đếm lùi để tạo trễ 500ms.
- Nếu tự tay viết Assembly tương đương (Tuần 2: `SBI`/`CBI` xen kẽ 2 vòng lặp trễ), kích thước và tốc độ gần như **giống hệt** — vì đoạn mã đủ đơn giản để trình biên dịch tối ưu hoàn hảo.
- **Kết luận sư phạm**: C không ”trả giá” về hiệu năng cho các thao tác GPIO/định thời cơ bản, nhưng giúp ta không phải tự tay tính toán vòng lặp trễ hay quản lý thanh ghi tạm — đó chính là giá trị thật của C trong hệ nhúng.

Kiểm tra nhanh — Câu hỏi

Thảo luận

1. Vì sao hàm `main()` trong chương trình nhúng không bao giờ dùng lệnh `return`?
2. Nêu đúng thứ tự 4 bước của luồng biên dịch avr-gcc, và cho biết công cụ nào thực hiện từng bước.

Kiểm tra nhanh — Gợi ý đáp án

1. Vì không có hệ điều hành nào để ”nhận lại” quyền điều khiển — vi điều khiển phải hoạt động liên tục trong khi còn cấp nguồn, nên `main()` luôn chứa vòng lặp vô hạn thay vì return.
2. (1) Tiền xử lý — `cpp` (mở rộng `#include`/`#define`); (2) Biên dịch — `avr-gcc` sinh hợp ngữ; (3) Hợp dịch — `avr-as` sinh file `.o`; (4) Liên kết — `avr-ld` gộp thành `.elf`, sau đó `avr-objcopy` tạo `.hex`.

Kiểu dữ liệu và biến trong C nhúng

Vì sao kiểu dữ liệu quan trọng trong lập trình nhúng?

- Arduino Uno (ATmega328P) chỉ có **2 KB SRAM** — chọn sai kiểu dữ liệu (dùng `int` 32-bit khi 8-bit là đủ) có thể lãng phí bộ nhớ quý giá gấp 4 lần.
- CPU AVR là bộ xử lý **8-bit**: các phép toán trên kiểu lớn hơn 8-bit tốn nhiều lệnh máy hơn hẳn — chọn đúng kiểu ảnh hưởng trực tiếp đến tốc độ.
- Khác với lập trình ứng dụng trên PC (bộ nhớ dồi dào), lập trình nhúng đòi hỏi **ý thức rõ ràng** về kích thước từng biến.

Kiểu dữ liệu chuẩn C trên AVR 8-bit

Kiểu	Kích thước	Phạm vi giá trị
`char`	1 byte	-128 .. 127
`unsigned char`	1 byte	0 .. 255
`int`	2 byte	-32768 .. 32767
`unsigned int`	2 byte	0 .. 65535
`long`	4 byte	$\approx -2.1 \times 10^9$ .. 2.1×10^9
`unsigned long`	4 byte	0 .. $\approx 4.3 \times 10^9$
`float` / `double`	4 byte	(double = float trên avr-gcc)

Đã xác minh bằng `sizeof()` thực tế với avr-gcc: `int` chỉ **16-bit** trên AVR — khác với 32-bit quen thuộc trên PC!

Kiểu có dấu (signed) và không dấu (unsigned)

- **Signed**: dành 1 bit cao nhất biểu diễn dấu ($-$/$+$), phạm vi lệch về phía âm.
- **Unsigned**: toàn bộ bit dùng cho độ lớn, luôn ≥ 0, phạm vi dương gấp đôi.
- Ví dụ tràn số (overflow): `uint8_t a = 200; uint8_t b = 100; uint8_t c = a + b;` $\rightarrow 300 \bmod 256 = 44$ — kết quả sai hoàn toàn nếu không lường trước.
- Với thanh ghi/bộ đếm phần cứng, gần như luôn dùng kiểu **unsigned** (không có giá trị âm về mặt vật lý).

Vấn đề: kích thước kiểu dữ liệu chuẩn không cố định

Không portable (không di động)

Chuẩn C chỉ đảm bảo `int` **tối thiểu** 16-bit — không quy định kích thước chính xác. Kích thước thật phụ thuộc trình biên dịch và kiến trúc CPU.

- Trên AVR (avr-gcc): `int` = 16-bit.
- Trên PC/ARM 32-bit thông thường: `int` = 32-bit.
- Hệ quả: mã viết `int` trên PC rồi đem chạy trên AVR có thể tràn số âm thầm — vì phạm vi giá trị nhỏ hơn hẳn dự kiến.

Kiểu dữ liệu chính xác cố định — stdint.h

stdint.h

Header chuẩn C99 định nghĩa các kiểu có **kích thước cố định, tường minh**, không phụ thuộc trình biên dịch/kiến trúc: `uint8_t`, `int16_t`, `uint32_t`...

- Tên kiểu tự mô tả: `uintN_t` = unsigned N-bit, `intN_t` = signed N-bit.
- Đây là lựa chọn **được khuyến nghị hàng đầu** trong mọi mã nguồn nhúng chuyên nghiệp (bao gồm chính avr-libc).

Bảng đầy đủ các kiểu trong stdint.h

Kiểu	Kích thước	Phạm vi giá trị
`uint8_t`	1 byte	0 .. 255
`int8_t`	1 byte	-128 .. 127
`uint16_t`	2 byte	0 .. 65535
`int16_t`	2 byte	-32768 .. 32767
`uint32_t`	4 byte	0 .. 4294967295
`int32_t`	4 byte	$\approx \pm 2.1 \times 10^9$
`uint64_t` / `int64_t`	8 byte	Hiếm dùng trên MCU 8-bit

Ví dụ code: khai báo biến với stdint.h

```c
#include <stdint.h>

uint8_t  so_lan_nhan_nut = 0;      // 0..255, du cho bo dem nut nhan
int16_t  nhiet_do_x10    = -50;    // vd: -5.0 do C, luu x10 de tranh so thuc
uint32_t so_ms_da_troi_qua = 0;    // bo dem thoi gian, can pham vi lon

void tang_bo_dem(void) {
    if (so_lan_nhan_nut < 255) {   // kiem tra truoc khi tang, tranh tran so
        so_lan_nhan_nut++;
    }
}
```

Quy ước: luôn khai báo biến với **kiểu nhỏ nhất đủ dùng** — tiết kiệm SRAM và tăng tốc độ xử lý trên CPU 8-bit.

Biến volatile — vấn đề tối ưu hoá của trình biên dịch

- Trình biên dịch tối ưu hoá bằng cách **giả định** một biến chỉ thay đổi khi chính mã đang xét ghi vào nó.
- Với thanh ghi phần cứng hoặc biến bị ngắt (ISR) sửa đổi, giả định này **sai**: giá trị có thể thay đổi bởi phần cứng/ngắt mà trình biên dịch không ”nhìn thấy” trong luồng mã tuyến tính.
- Hệ quả nếu không cảnh báo: trình biên dịch có thể đọc biến **một lần rồi giữ trong thanh ghi CPU mãi mãi**, bỏ qua các lần đọc lại tưởng chừng dư thừa — gây lỗi logic nghiêm trọng, rất khó debug.
- Giải pháp: từ khoá `volatile` báo cho trình biên dịch ”luôn đọc/ghi biến này từ bộ nhớ thật, không được tối ưu hoá bỏ qua”.

Ví dụ lỗi: thiếu từ khoá volatile

```
uint8_t flag = 0;              // LOI: thieu volatile

ISR(INT0_vect) {
    flag = 1;                  // ngat ngoai set co
}

int main(void) {
    while (flag == 0) {        // cho ngat xay ra
        ;
    }
    PORTB |= (1 << PB0);       // KHONG BAO GIO chay toi day!
    while (1);
}
```

Vì sao lỗi? Bằng chứng từ avr-objdump (-0s)

```
main:                       ; flag KHONG volatile
   LDS  r24, flag            ; doc flag DUY NHAT 1 LAN
   CPSE r24, r1               ; so sanh voi 0
   RJMP .+2                   ; neu != 0, thoat vong lap
   RJMP .-2                   ; neu == 0, LAP VO HAN TAI DAY!
```

Đây là mã máy thật, đã biên dịch và kiểm chứng

Vòng lặp RJMP .-2 chỉ nhảy vào chính nó — CPU **không bao giờ đọc lại** flag từ bộ nhớ, dù ISR có set flag = 1 sau đó. Chương trình treo vĩnh viễn.

Ví dụ đúng: dùng volatile cho biến chia sẻ với ISR

```
volatile uint8_t flag = 0;     // DUNG: co volatile

ISR(INT0_vect) {
    flag = 1;
}

int main(void) {
    while (flag == 0) {        // luon doc lai flag tu bo nho
        ;
    }
    PORTB |= (1 << PB0);       // chay dung khi ngat xay ra
    while (1);
}
```

Mã máy tương ứng (đã kiểm chứng) dùng BREQ nhảy **về lại lệnh LDS** — đọc lại flag mỗi vòng lặp, phát hiện đúng khi ISR thay đổi giá trị.

Quy tắc: khi nào bắt buộc dùng volatile

- Biến ánh xạ trực tiếp thanh ghi phần cứng (thường đã có sẵn `volatile` trong `avr/io.h`, không cần khai báo lại).
- Biến toàn cục được **đọc trong vòng lặp chính** nhưng **ghi trong ISR** (hoặc ngược lại) — trường hợp phổ biến nhất.
- Biến dùng trong vòng lặp chờ bận (busy-wait) chờ một điều kiện phần cứng/ngắt thay đổi.
- **Không lạm dụng**: `volatile` ngăn mọi tối ưu hoá trên biến đó, làm chậm mã không cần thiết nếu dùng tràn lan cho biến không thực sự chia sẻ.

Từ khoá const trong C nhúng

- `const` báo cho trình biên dịch: biến **chỉ đọc**, không được phép ghi sau khi khởi tạo.
- Lợi ích: trình biên dịch có thể tối ưu hoá tốt hơn, cảnh báo/lỗi biên dịch nếu code lỡ tay ghi vào biến chỉ đọc, giúp phát hiện lỗi sớm.
- Dùng phổ biến cho: bảng tra cứu (lookup table), chuỗi thông báo, hệ số cấu hình cố định.
- Ví dụ: `const uint8_t MAX_RETRY = 5;`

Vấn đề: const vẫn chiếm SRAM trên AVR

Điểm khác biệt quan trọng trên AVR

Do kiến trúc Harvard (bộ nhớ chương trình và dữ liệu tách biệt), theo mặc định **mọi biến `const` vẫn được sao chép vào SRAM** lúc khởi động — dù giá trị không đổi!

- Khác với PC (bộ nhớ chung, `const` có thể nằm thẳng trong vùng chỉ đọc của chương trình).
- Với SRAM chỉ 2 KB trên ATmega328P, một bảng hằng số lớn khai báo `const` thông thường có thể chiếm hết bộ nhớ quý giá.
- Giải pháp: chỉ thị `PROGMEM` — trình bày ở slide tiếp theo.

PROGMEM — lưu hằng số trực tiếp trong Flash

avr/pgmspace.h

Chỉ thị `PROGMEM` yêu cầu trình biên dịch đặt dữ liệu **cố định trong bộ nhớ Flash**, không sao chép vào SRAM — tận dụng Flash 32 KB thay vì SRAM 2 KB.

- Vì kiến trúc Harvard tách biệt 2 loại bộ nhớ, CPU **không thể đọc trực tiếp** Flash bằng lệnh thường — phải dùng hàm chuyên dụng `pgm_read_byte()`, `pgm_read_word()`.
- Đánh đổi: đọc chậm hơn SRAM một chút, nhưng tiết kiệm SRAM đáng kể cho dữ liệu lớn, ít thay đổi.

Ví dụ code: bảng tra cứu với PROGMEM

```
#include <avr/pgmspace.h>

// Bang nam trong Flash, KHONG chiem SRAM
const uint8_t bang_led[] PROGMEM = {
    0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80
};

uint8_t doc_gia_tri(uint8_t i) {
    return pgm_read_byte(&bang_led[i]);   // doc tu Flash
}
```

Đã kiểm chứng bằng avr-gcc: đoạn mã trên biên dịch sạch, không cảnh báo — cú pháp chuẩn của avr-libc.

Toán tử bit-wise: AND, OR, XOR, NOT

Toán tử	Ký hiệu C	Ý nghĩa	Bảng chân trị (1 bit)
AND	&	Cả 2 bit đều 1	$1\&1=1$; còn lại $=0$
OR	|	Ít nhất 1 bit là 1	$0\|0=0$; còn lại $=1$
XOR	^	Hai bit khác nhau	$1\oplus0=1$; $1\oplus1=0$
NOT	~	Đảo ngược từng bit	$\sim1=0$; $\sim0=1$

Đây chính là 4 phép toán logic mà phần cứng ALU của AVR thực hiện trực tiếp bằng 1 lệnh máy (`AND`, `OR`, `EOR`, `COM`) mỗi loại.

Toán tử dịch bit: Left shift và Right shift

- `x << n`: dịch trái n bit — các bit thấp được lấp đầy bằng 0. Tương đương nhân x với 2^n.
- `x >> n`: dịch phải n bit — tương đương chia x cho 2^n (với số không dấu).
- Dùng phổ biến nhất trong nhúng: tạo *mask* chọn đúng 1 bit — `(1 << PB5)` tạo giá trị nhị phân chỉ có bit số 5 là 1.
- Dịch bit rẻ hơn nhân/chia thông thường rất nhiều trên AVR — sẽ phân tích chi tiết ở Chương 3.

Kỹ thuật SET một bit

```
// SET bit "bit" cua thanh ghi "reg" ve 1, giu nguyen cac bit khac
#define SET_BIT(reg, bit)   ((reg) |= (1 << (bit)))

// Vi du:
SET_BIT(PORTB, PB5);   // <=> PORTB |= (1 << PB5);
```

- Cơ chế: `1 << bit` tạo mask chỉ có 1 bit cần set là 1, các bit còn lại là 0.
- Phép OR với mask: bit cần set chắc chắn thành 1; các bit khác giữ nguyên (vì `X | 0 = X`).
- Trình biên dịch avr-gcc dịch mẫu hình này thành đúng 1 lệnh máy `SBI` khi có thể (đã kiểm chứng ở phần trước).

Kỹ thuật CLEAR một bit

```
// CLEAR bit "bit" cua thanh ghi "reg" ve 0, giu nguyen cac bit khac
#define CLEAR_BIT(reg, bit) ((reg) &= ~(1 << (bit)))

// Vi du:
CLEAR_BIT(PORTB, PB5);  // <=> PORTB &= ~(1 << PB5);
```

- Cơ chế: `(1 << bit)` tạo mask có **mọi bit là 1 ngoại trừ** bit cần xoá (bit đó là 0).
- Phép `AND` với mask: bit cần xoá chắc chắn thành 0; các bit khác giữ nguyên (vì `X & 1 = X`).
- Trình biên dịch dịch mẫu hình này thành đúng 1 lệnh máy `CBI`.

Kỹ thuật TOGGLE và kiểm tra một bit

```
// TOGGLE (dao) bit "bit": 0->1 hoac 1->0
#define TOGGLE_BIT(reg, bit) ((reg) ^= (1 << (bit)))

// KIEM TRA bit "bit" co dang la 1 hay khong (khac 0 neu dung)
#define CHECK_BIT(reg, bit)  ((reg) & (1 << (bit)))

if (CHECK_BIT(PINB, PB0)) {   // neu nut nhan dang o muc cao
    TOGGLE_BIT(PORTB, PB5);   // dao trang thai LED
}
```

Mẹo phần cứng AVR: ghi 1 vào bit tương ứng của thanh ghi `PINx` (không phải `PORTx`) cũng tạo hiệu ứng toggle — chỉ tốn 2 lệnh máy thay vì 4.

Toán tử gán phức hợp

```
uint8_t reg = 0;

reg |= (1 << 3);   // tuong duong: reg = reg | (1 << 3);
reg &= ~(1 << 3);  // tuong duong: reg = reg & ~(1 << 3);
reg ^= (1 << 3);   // tuong duong: reg = reg ^ (1 << 3);
reg <<= 2;         // tuong duong: reg = reg << 2;
reg >>= 1;         // tuong duong: reg = reg >> 1;
```

- Cú pháp rút gọn op= tồn tại cho mọi toán tử hai ngôi: +=, -=, |=, &=, ^=, <<=, >>=...
- Đây chính là cú pháp thực tế dùng trong `DDRB |= (1 << PB5);`, `PORTB &= ~(1 << PB5);` mà ta đã dùng từ Tuần 1.

Macro tiền xử lý #define — khái niệm

- `#define` định nghĩa một **phép thay thế văn bản thuần tuý**, được bộ tiền xử lý thực hiện **trước khi** trình biên dịch thấy mã nguồn.
- Dạng đơn giản nhất — hằng số đặt tên: `#define MAX_RETRY 5` → mọi chỗ dùng `MAX_RETRY` được thay bằng văn bản `5`.
- Không tốn bộ nhớ chạy (runtime), không có kiểu dữ liệu, không có phạm vi (scope) như biến — thuần tuý là thao tác văn bản.
- Được dùng rất nhiều trong avr-libc: tên thanh ghi, tên bit đều là macro trỏ tới địa chỉ vật lý.

So sánh #define và hàm (function)

Tiêu chí	Macro (#define)	Hàm (function)
Chi phí gọi (overhead)	Không — thay thế văn bản trực tiếp	Có — CALL/RET, lưu ngăn xếp
Kiểm tra kiểu dữ liệu	Không	Có — trình biên dịch kiểm tra
Dễ debug	Khó — không thấy trong debugger như 1 hàm	Dễ — có thể đặt breakpoint
Rủi ro tác dụng phụ	Cao nếu viết sai (xem slide sau)	Thấp
Kích thước mã	Có thể phình to nếu dùng nhiều lần	Cố định, dùng lại 1 chỗ trong bộ nhớ

Macro có tham số (function-like macro)

```
#define MAX(a, b)  ((a) > (b) ? (a) : (b))
#define MIN(a, b)  ((a) < (b) ? (a) : (b))

uint8_t x = MAX(5, 10);     // mo rong thanh: ((5) > (10) ? (5) : (10))
uint8_t y = MIN(cam_bien, 100);
```

- Cú pháp giống lời gọi hàm, nhưng vẫn là thay thế văn bản — không có kiểu tham số cố định (hoạt động với mọi kiểu số).
- Rất phổ biến cho các thao tác ngắn, lặp lại nhiều: `SET_BIT`, `MAX`, `MIN`, chuyển đổi đơn vị đơn giản.

Lỗi thường gặp: thiếu ngoặc bảo vệ trong macro

```
#define SQUARE_BAD(x)  x*x        // LOI: thieu ngoac

int a = 25 / SQUARE_BAD(2+3);
// Mo rong THUC TE:  25 / 2+3*2+3
// = (25/2) + (3*2) + 3 = 12 + 6 + 3 = 21   <-- SAI!
// Ky vong: 25 / (5*5) = 25/25 = 1
```

Đã kiểm chứng bằng trình biên dịch thật

Biểu thức trên thực sự cho kết quả **21** thay vì **1** — vì `#define` chỉ thay thế văn bản, **không** tự động thêm ngoặc quanh tham số hay toàn biểu thức.

Cách viết macro an toàn

```
#define SQUARE_GOOD(x)  ((x) * (x))    // DUNG: co ngoac day du

int b = 25 / SQUARE_GOOD(2+3);
// Mo rong: 25 / ((2+3) * (2+3)) = 25 / (5*5) = 25/25 = 1  <-- DUNG
```

Quy tắc vàng viết macro

1. Luôn bọc **từng tham số** trong ngoặc đơn: `(x)` chứ không phải x.
2. Luôn bọc **toàn bộ biểu thức** kết quả trong ngoặc đơn.
3. Cân nhắc dùng hàm `inline` (C99) khi cần an toàn kiểu dữ liệu hơn.

Hàm trong C: khai báo và định nghĩa

```c
// Khai bao (prototype) - thuong dat trong file .h
uint16_t doc_cam_bien(uint8_t kenh);

// Dinh nghia (definition) - than ham, dat trong file .c
uint16_t doc_cam_bien(uint8_t kenh) {
    // ... doc gia tri ADC tu kenh chi dinh ...
    return gia_tri;
}
```

- **Khai báo**: cho trình biên dịch biết ”chữ ký” hàm (tên, kiểu trả về, kiểu tham số) để dùng ở nơi khác trước khi thấy định nghĩa đầy đủ.
- **Định nghĩa**: cung cấp phần thân hàm — mã máy thực sự sẽ được sinh ra.

Truyền tham số cho hàm: by value vs by pointer

```
void tang_gia_tri_value(uint8_t x) {
    x = x + 1;              // chi sua BAN SAO cuc bo, khong anh huong ben ngoai
}

void tang_gia_tri_pointer(uint8_t *x) {
    *x = *x + 1;             // sua truc tiep qua DIA CHI, anh huong ben ngoai
}

uint8_t bien = 5;
tang_gia_tri_value(bien);    // bien van la 5 sau khi goi
tang_gia_tri_pointer(&bien); // bien tro thanh 6 sau khi goi
```

Truyền theo tham chiếu (con trỏ) cần thiết khi hàm phải **trả về nhiều giá trị** hoặc sửa dữ liệu gốc mà không tốn chi phí sao chép.

Con trỏ (pointer) cơ bản và truy cập thanh ghi phần cứng

```
// avr/io.h dinh nghia thanh ghi theo dung co che nay (don gian hoa):
#define PORTB (*(volatile uint8_t *)0x25)

// PORTB thuc chat la MOT DIA CHI BO NHO co dinh
// Ghi vao PORTB = ghi truc tiep vao dia chi 0x25
PORTB = 0xFF;
```

- Con trỏ lưu **địa chỉ** của một ô nhớ, không lưu trực tiếp giá trị.
- Bản chất mọi thanh ghi phần cứng AVR (DDRB, PORTB...) chỉ là các **địa chỉ bộ nhớ cố định** — avr-libc dùng con trỏ `volatile` để ánh xạ tên gợi nhớ tới đúng địa chỉ vật lý.

Biên dịch riêng biệt: tổ chức file .c / .h

- Chương trình lớn được chia thành nhiều file `.c` (mỗi file biên dịch độc lập thành 1 file `.o`) để dễ tổ chức và biên dịch lại nhanh hơn (chỉ file thay đổi mới cần biên dịch lại).
- File `.h` (header) chứa các **khai báo dùng chung**: prototype hàm, định nghĩa `struct`, macro — được `#include` vào mọi file `.c` cần dùng.
- Từ khoá `extern` khai báo ”biến/hàm này được định nghĩa ở file khác” — cho phép nhiều file `.c` chia sẻ cùng 1 biến toàn cục hoặc hàm.
- *Header guard* (`#ifndef`/`#define`/`#endif`) ngăn một file `.h` bị `#include` lặp lại nhiều lần gây lỗi định nghĩa trùng.

Ví dụ: biên dịch riêng biệt

```c
// -- sensor.h --
#ifndef SENSOR_H
#define SENSOR_H
#include <stdint.h>
uint16_t doc_cam_bien(uint8_t kenh);  // khai bao (prototype)
#endif
// -- sensor.c --
#include "sensor.h"
uint16_t doc_cam_bien(uint8_t kenh) { return 0; }
// -- main.c --
#include "sensor.h"        // biet prototype de goi ham
int main(void) {
    uint16_t v = doc_cam_bien(0);
    while (1) {}
}
```

Biên dịch riêng `sensor.c` và `main.c` thành 2 file `.o`, rồi liên kết lại thành 1 chương trình.

Struct — mô tả dữ liệu có cấu trúc

```c
typedef struct {
    uint16_t gia_tri_adc;
    uint8_t  kenh;
    uint8_t  hop_le;   // 1 = du lieu hop le, 0 = chua doc
} CamBien_t;

CamBien_t cb1 = {0, 0, 0};

void cap_nhat(CamBien_t *cb, uint16_t gt) {
    cb->gia_tri_adc = gt;    // toan tu -> truy cap qua con tro struct
    cb->hop_le = 1;
}
```

`struct` nhóm nhiều biến liên quan thành 1 kiểu dữ liệu — rất hữu ích để mô tả trạng thái cảm biến, gói dữ liệu UART, hoặc bố cục thanh ghi phần cứng.

Phạm vi biến (scope): global, local, static

- **Biến toàn cục (global)**: khai báo ngoài mọi hàm, tồn tại suốt vòng đời chương trình, nằm ở vùng `.data`/`.bss` trong SRAM.
- **Biến cục bộ (local)**: khai báo trong hàm, chỉ tồn tại khi hàm đang thực thi, nằm trên *ngăn xếp* (stack) — mất giá trị khi hàm kết thúc.
- **Biến `static` cục bộ**: khai báo trong hàm nhưng **giữ nguyên giá trị** giữa các lần gọi — nằm ở `.data`/`.bss` như biến toàn cục, nhưng chỉ hàm đó nhìn thấy được.
- Nguyên tắc: hạn chế biến toàn cục khi có thể — dùng tham số/con trỏ để mã dễ kiểm soát và tái sử dụng hơn.

Ép kiểu (type casting) và nguy cơ mất dữ liệu

```
uint16_t lon = 300;
uint8_t  nho = (uint8_t)lon;    // ep kieu tuong minh: 300 -> 44 (mat du lieu!)

int16_t  x = -1;
uint16_t y = 1;
if (x < y) { /* MONG DOI: vao day vi -1 < 1 */ }
else       { /* THUC TE tren AVR: x bi ep NGAM DINH thanh unsigned (65535) */ }
// Da kiem chung: 65535 < 1 la SAI -> chuong trinh vao nhanh else!
```

Cẩn trọng khi so sánh signed với unsigned

C tự động ép kiểu ngầm định (implicit conversion) khi so sánh 2 kiểu khác dấu cùng hạng — kết quả có thể trái trực giác. Luôn để ý kiểu dữ liệu khi viết điều kiện so sánh.

Kiểm tra nhanh — Câu hỏi

Thảo luận

1. Vì sao biến cờ (flag) được ISR ghi và vòng lặp chính đọc bắt buộc phải khai báo `volatile`?
2. Macro `#define DOUBLE(x) x+x` có an toàn không? Nếu gọi `DOUBLE(3+4)` thì kết quả thực tế là bao nhiêu?

Kiểm tra nhanh — Gợi ý đáp án

1. Không có `volatile`, trình biên dịch có thể chỉ đọc biến 1 lần rồi giữ trong thanh ghi CPU, không nhận ra ISR đã thay đổi giá trị trong bộ nhớ — gây treo chương trình vĩnh viễn (đã minh hoạ bằng mã máy thật ở phần trước).
2. Không an toàn — thiếu ngoặc bảo vệ. `DOUBLE(3+4)` mở rộng thành `3+4+3+4 = 14`, không phải `2*(3+4)=14`... nhưng thử `10/DOUBLE(3+4)` sẽ lộ lỗi: mở rộng thành `10/3+4+3+4=3+4+3+4=14` thay vì `10/14`. Cách sửa: `#define DOUBLE(x) ((x)+(x))`.

Tổng kết: Kiểu dữ liệu và biến trong C nhúng

- Luôn ưu tiên kiểu chính xác cố định (`stdint.h`) thay vì `int`/`long` chuẩn — tường minh và di động hơn.
- `volatile` bắt buộc cho thanh ghi phần cứng và biến chia sẻ với ISR; `PROGMEM` giúp tiết kiệm SRAM cho hằng số lớn.
- Toán tử bit-wise (&, |, ^, ~, <<, >>) là công cụ cốt lõi để thao tác thanh ghi từng bit.
- Macro tiết kiệm chi phí gọi hàm nhưng cần ngoặc bảo vệ nghiêm ngặt; hàm an toàn kiểu hơn nhưng có overhead.
- Tổ chức mã theo `.c`/`.h` với `extern` giúp chương trình lớn dễ quản lý và biên dịch lại nhanh.

Số học và giới hạn của C trên vi điều khiển 8-bit

Giới hạn ALU 8-bit của AVR

- ALU (Arithmetic Logic Unit) của ATmega328P xử lý **tối đa 8 bit trong 1 lệnh**, dù ngôn ngữ C cho phép khai báo biến 16, 32, thậm chí 64-bit.
- Với biến lớn hơn 8-bit, trình biên dịch phải **tự động chia nhỏ** phép toán thành nhiều thao tác trên từng byte — người lập trình C không thấy điều này, nhưng nó ảnh hưởng trực tiếp đến tốc độ và kích thước mã.
- Hiểu rõ chi phí này giúp lập trình viên nhúng chọn đúng kiểu dữ liệu và tránh các phép toán đắt đỏ không cần thiết.

Số học đa độ chính xác (Multi-precision Arithmetic) — khái niệm

Multi-precision arithmetic

Kỹ thuật biểu diễn và tính toán trên số có độ rộng **lớn hơn** độ rộng thanh ghi/ALU của CPU, bằng cách ghép nhiều số nhỏ hơn (thường theo byte) lại với nhau.

- CPU 8-bit + số 16-bit $\rightarrow$ số được lưu thành 2 byte (byte thấp + byte cao), xử lý tuần tự từng byte kèm theo dõi *carry* (nhớ) giữa các byte.
- Đây là điều mà Assembly Tuần 2 phải làm **thủ công**; trình biên dịch C làm việc này **tự động và chính xác**.

Cộng số 16-bit trên AVR 8-bit — carry propagation

- Số 16-bit được lưu thành 2 byte: byte thấp (low byte) và byte cao (high byte).
- Bước 1: cộng 2 byte thấp bằng lệnh `ADD` — nếu kết quả tràn quá 255, cờ `Carry` (C) trong thanh ghi `SREG` được set lên 1.
- Bước 2: cộng 2 byte cao bằng lệnh `ADC` (Add with Carry) — **cộng thêm cả cờ Carry** từ bước 1 vào kết quả.
- Cơ chế ”lan truyền nhớ” (carry propagation) này giống hệt việc cộng tay 2 số nhiều chữ số: cộng từ hàng thấp nhất, nhớ sang hàng kế tiếp.

Ví dụ Assembly: cộng hai số 16-bit

```
; Cong so 16-bit: R25:R24 = R25:R24 + R23:R22
ADD  R24, R22      ; cong byte THAP, cap nhat co Carry
ADC  R25, R23      ; cong byte CAO, CONG THEM co Carry tu buoc truoc
```

Đã kiểm chứng: hàm C `uint16_t add16(uint16_t a, uint16_t b) { return a+b; }` biên dịch (`-Os`) ra **đúng 2 lệnh** `ADD` + `ADC` này — không thừa một lệnh nào.

Cộng/trừ số 32-bit — mở rộng carry propagation

- Số 32-bit gồm 4 byte — nguyên lý carry propagation lặp lại: 1 lệnh `ADD` cho byte thấp nhất, rồi **3 lệnh `ADC`** liên tiếp cho 3 byte còn lại.
- Đã kiểm chứng: hàm C `uint32_t add32(uint32_t a, uint32_t b)` biên dịch ra **chính xác 4 lệnh**: `ADD, ADC, ADC, ADC`.
- Quy luật tổng quát: cộng số N byte cần N lệnh (1 `ADD` + ($N-1$) `ADC`) — chi phí tăng **tuyến tính** theo kích thước kiểu dữ liệu.

C tự động hoá số học đa độ chính xác

Giá trị thực sự của C ở đây

Người lập trình C chỉ cần viết `c = a + b;` với `a, b, c` kiểu `uint32_t` — trình biên dịch **tự sinh đúng** chuỗi `ADD`+`ADC`×3, không cần người lập trình tự theo dõi cờ Carry thủ công.

- Đây chính là lý do C được ưa chuộng: loại bỏ hoàn toàn nguy cơ lỗi do quên xử lý carry khi làm việc với số nhiều byte.
- Không có ”chi phí ẩn” bất ngờ — mã C tối ưu sinh ra đúng số lệnh tối thiểu cần thiết, như đã kiểm chứng ở 2 slide trước.

Phép nhân số nguyên trên AVR

- ATmega328P có **bộ nhân phần cứng** (hardware multiplier) 8×8-bit → 16-bit — thực hiện bằng lệnh `MUL` duy nhất.
- Đã kiểm chứng: `uint16_t mul8x8(uint8_t a, uint8_t b) { return (uint16_t)a*b; }` chỉ cần **3 lệnh máy** (`MUL` + `MOVW` + `EOR`) — rất rẻ.
- Đây là điểm khác biệt so với nhiều vi điều khiển 8-bit giá rẻ không có bộ nhân phần cứng (phải nhân bằng vòng lặp cộng dồn).

Chi phí phép nhân 16-bit và 32-bit

- Nhân 2 số 16-bit không thể thực hiện bằng 1 lệnh `MUL` (chỉ hỗ trợ 8×8) — trình biên dịch phải **chia thành nhiều phép nhân 8-bit từng phần rồi cộng dồn**.
- Đã kiểm chứng: `uint16_t mul16(uint16_t a, uint16_t b)` biên dịch ra **9 lệnh máy** (3×`MUL` + `ADD`×2 + `MOVW`×2 + `EOR`) — gấp 3 lần chi phí nhân 8-bit.
- Nhân 32-bit còn tốn kém hơn nhiều (nhiều phép nhân 8-bit từng phần hơn nữa) — nên tránh nếu có thể thay bằng kiểu nhỏ hơn hoặc dịch bit.

Phép chia số nguyên trên AVR

AVR không có lệnh chia phần cứng!

Khác với phép nhân, ATmega328P **không có** lệnh chia (*divide*) trong tập lệnh. Mọi phép chia số nguyên phải thực hiện bằng **hàm thư viện phần mềm**.

- Đã kiểm chứng: `uint8_t div8(uint8_t a, uint8_t b) { return a/b; }` biên dịch thành 1 lệnh `CALL` tới hàm thư viện `__udivmodqi4`.
- Bên trong `__udivmodqi4` là một **vòng lặp dịch-trừ** (shift-subtract) chạy 8–9 lần lặp — đắt hơn hẳn 1 lệnh `ADD` đơn thuần.

Bảng so sánh chi phí các phép toán số học nguyên

Phép toán	8-bit	16-bit	32-bit
Cộng/trừ	1 lệnh	2 lệnh	4 lệnh
Nhân	1 lệnh (`MUL` phần cứng)	9 lệnh (đã đo)	Hàng chục lệnh
Chia	Gọi hàm thư viện (∼8–9 vòng lặp)	Gọi hàm thư viện (đắt hơn)	Rất đắt (vòng lặp dài)

Số liệu cột 8/16-bit đã được kiểm chứng trực tiếp bằng `avr-gcc -Os` + `avr-objdump`.

Quy tắc tối ưu: thay nhân/chia luỹ thừa 2 bằng dịch bit

- Nếu số chia/nhân là **luỹ thừa của 2** $(2, 4, 8, 16...)$, dùng dịch bit thay vì toán tử */// thông thường.
- Ví dụ: `x / 4` $\rightarrow$ `x >> 2`; `x * 8` $\rightarrow$ `x << 3`.
- Với hằng số luỹ thừa 2, avr-gcc **tự động** thực hiện tối ưu hoá này ngay cả khi ta viết x/4 — nhưng viết tường minh x>>2 giúp mã rõ ràng và chắc chắn tối ưu ngay cả với trình biên dịch kém tối ưu hơn.
- Với số chia **không phải** luỹ thừa 2, không có cách nào tránh chi phí chia — cân nhắc thiết kế lại thuật toán nếu có thể.

Số dấu chấm động (float) trên AVR

Không có FPU phần cứng

ATmega328P **không có bộ xử lý dấu chấm động** (FPU — Floating Point Unit) phần cứng. Mọi phép toán `float` phải thực hiện bằng **thư viện phần mềm** (software floating-point library) của avr-libc.

- `float` trên AVR có kích thước 4 byte theo chuẩn IEEE 754 đơn (single precision) — nhưng **mọi phép toán** đều tốn nhiều lệnh máy hơn số nguyên rất nhiều.
- `double` trên avr-gcc thực chất **giống hệt `float`** (cũng 4 byte) — đã kiểm chứng bằng `sizeof()`.

Chi phí thư viện float phần mềm

- Đã kiểm chứng: hàm `float addf(float a, float b) { return a+b; }` biên dịch thành 1 lệnh `CALL` tới `__addsf3` — nhưng hàm này lại gọi tiếp `__addsf3x` có kích thước **204 byte** (khoảng 100 lệnh máy!) cùng nhiều hàm phụ trợ (`__fp_split3`, `__fp_round`...).
- So sánh trực tiếp: chương trình LED nhấp nháy đơn giản chỉ **162 byte** mã máy; thêm đúng 1 phép chia 8-bit + 1 phép cộng float làm chương trình tăng lên **578 byte** — gấp **3,6 lần**!
- Một phép cộng `float` đơn giản có thể tốn **hàng chục đến hàng trăm chu kỳ xung nhịp**, so với 1 chu kỳ của phép cộng số nguyên 8-bit.

Hệ quả: khi nào nên/không nên dùng float

- **Tránh** `float` trong: vòng lặp tần suất cao (ISR, vòng lặp chính chạy liên tục), hệ thống thời gian thực chặt, ứng dụng cực kỳ giới hạn Flash/tốc độ.
- **Có thể chấp nhận** `float` khi: tính toán không thường xuyên (một lần khi khởi động, hiển thị kết quả cuối), độ phức tạp thuật toán không đáng kể so với chi phí phát triển nếu tự làm fixed-point.
- Giải pháp thay thế phổ biến nhất trong công nghiệp: **fixed-point arithmetic** — trình bày ở slide tiếp theo.

Kỹ thuật thay thế: Fixed-point Arithmetic — khái niệm Q-format

Fixed-point arithmetic

Biểu diễn số thực bằng số **nguyên**, quy ước trước một số bit dành cho phần thập phân — tính toán hoàn toàn bằng phép toán số nguyên (rẻ), không cần thư viện float.

- Ký hiệu **Qm.n**: m bit phần nguyên, n bit phần thập phân. Giá trị thực = số nguyên lưu trữ $\div 2^n$.
- Ví dụ Q8.8 (dùng `int16_t`): 8 bit nguyên + 8 bit thập phân $\rightarrow$ chia cho $2^8 = 256$ để lấy giá trị thực.

Ví dụ Q-format ngắn gọn (Q8.8)

```
typedef int16_t fixed_t;              // Q8.8: 8 bit nguyen, 8 bit thap phan
#define FIX_SHIFT      8
#define TO_FIXED(x)    ((fixed_t)((x) * 256))
#define FIXED_MUL(a,b) ((fixed_t)(((int32_t)(a) * (b)) >> FIX_SHIFT))

fixed_t gia_tri  = TO_FIXED(3.5);              // 3.5 -> 896 (0x0380)
fixed_t ket_qua  = gia_tri + TO_FIXED(2.0);    // cong: chi 1 phep ADC 16-bit!
fixed_t tich     = FIXED_MUL(gia_tri, TO_FIXED(2));  // 3.5*2 = 7.0
```

Phép **cộng** fixed-point = cộng số nguyên bình thường (rẻ); phép **nhân** cần dịch phải sau khi nhân để đưa về đúng thang đo.

Vì sao tránh đệ quy (recursion) trong lập trình nhúng

- Mỗi lần gọi hàm đệ quy, CPU phải **lưu ngữ cảnh** (thanh ghi, địa chỉ trả về) lên ngăn xếp (stack) trước khi gọi tiếp — đã kiểm chứng: hàm Fibonacci đệ quy trên AVR cần PUSH 6 thanh ghi (12 byte) mỗi lần gọi.
- SRAM của ATmega328P chỉ có **2 KB** — với đệ quy sâu (nhiều lời gọi lồng nhau), ngăn xếp có thể **tràn** (stack overflow), ghi đè lên vùng nhớ khác và gây lỗi khó chẩn đoán.
- Độ sâu đệ quy **không kiểm soát được dễ dàng** tại thời điểm biên dịch — rủi ro khó lường trước trong hệ thống cần độ tin cậy cao.

Ví dụ: chuyển đệ quy thành vòng lặp

```
// DE QUY - ton PUSH/POP nhieu thanh ghi + CALL/RET moi lan goi
uint16_t giai_thua_dequy(uint8_t n) {
    if (n <= 1) return 1;
    return n * giai_thua_dequy(n - 1);  // goi lai chinh no
}
// LAP - khong ton chi phi ngan xep
uint16_t giai_thua_lap(uint8_t n) {
    uint16_t ket_qua = 1;
    for (uint8_t i = 2; i <= n; i++) {
        ket_qua *= i;
    }
    return ket_qua;
}
```

Quy tắc thực hành: **luôn ưu tiên vòng lặp** khi thuật toán cho phép — chỉ dùng đệ quy khi độ sâu được giới hạn chắc chắn.

Vì sao tránh cấp phát động malloc/free

- **Phân mảnh bộ nhớ** (fragmentation): cấp phát/giải phóng liên tục theo thời gian tạo ra các khoảng trống nhỏ rời rạc, có thể khiến một yêu cầu cấp phát lớn thất bại dù tổng bộ nhớ trống vẫn đủ.
- **Không xác định được thời gian** (non-deterministic): thời gian thực thi `malloc()` có thể thay đổi tuỳ tình trạng heap — vi phạm yêu cầu thời gian thực (real-time).
- SRAM chỉ 2 KB — không gian cho *heap* vốn đã rất hạn hẹp, chia sẻ với ngăn xếp (stack) đang phát triển ngược chiều.
- Rủi ro **rò rỉ bộ nhớ** (memory leak) nếu quên `free()` — trong hệ thống chạy liên tục nhiều năm, rò rỉ nhỏ cũng dẫn đến hỏng hệ thống sau cùng.

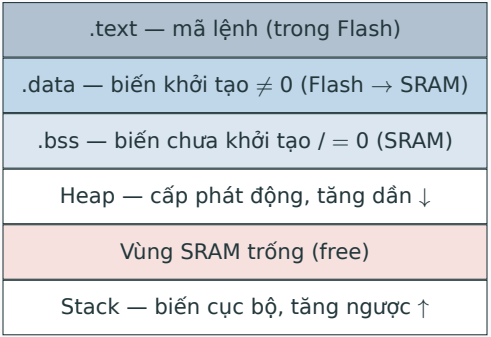
Giải pháp thay thế cấp phát động

- **Cấp phát tĩnh (static allocation)**: khai báo mảng/biến với kích thước cố định ngay tại thời điểm biên dịch — bộ nhớ được dành sẵn, không đổi trong suốt vòng đời chương trình.
- **Memory pool** (bể nhớ cố định): cấp phát trước 1 mảng lớn chia thành các khối kích thước cố định, tự quản lý cấp/thu hồi khối trong mảng đó — tránh phân mảnh, thời gian cấp phát xác định.
- Nguyên tắc chung trong firmware chuyên nghiệp (ô tô, y tế, hàng không): **cấm hoàn toàn** `malloc()/free()` sau giai đoạn khởi tạo, đôi khi cấm tuyệt đối trong toàn bộ chương trình.

Tổ chức bộ nhớ chương trình C trên AVR

- `.text` — mã lệnh chương trình (machine code), nằm trong bộ nhớ **Flash**, không đổi khi chạy.
- `.data` — biến toàn cục/static **có giá trị khởi tạo khác 0**: giá trị ban đầu lưu trong Flash, được sao chép vào SRAM lúc khởi động.
- `.bss` — biến toàn cục/static **không khởi tạo hoặc khởi tạo bằng 0**: chỉ cần cấp chỗ trong SRAM, tự động gán 0 lúc khởi động, không tốn Flash lưu giá trị.
- **Heap** — vùng cấp phát động (nếu dùng `malloc`), phát triển **tăng dần** từ sau `.bss`.
- **Stack** — ngăn xếp chứa biến cục bộ, tham số hàm, địa chỉ trả về, phát triển **giảm dần** từ đỉnh SRAM xuống.

Sơ đồ bộ nhớ chương trình C



Nếu Heap và Stack ”gặp nhau” khi cả hai cùng phát triển → tràn bộ nhớ (memory corruption) — nguy cơ tăng cao khi lạm dụng đệ quy hoặc cấp phát động trên SRAM 2 KB.

Bài tập nhỏ: ước lượng chi phí biểu thức số học

Đề bài

Cho đoạn mã: `uint32_t a, b, c; float f; … c = a + b; f = f / 3.0;` Hãy sắp xếp 2 phép toán trên theo thứ tự **chi phí tăng dần** và giải thích ngắn gọn.

- Gợi ý: xem lại số lệnh máy đo được cho phép cộng 32-bit so với phép toán liên quan đến `float` (chia) ở các slide trước.
- Đáp án được thảo luận trực tiếp trên lớp.

Kiểm tra nhanh — Câu hỏi

Thảo luận

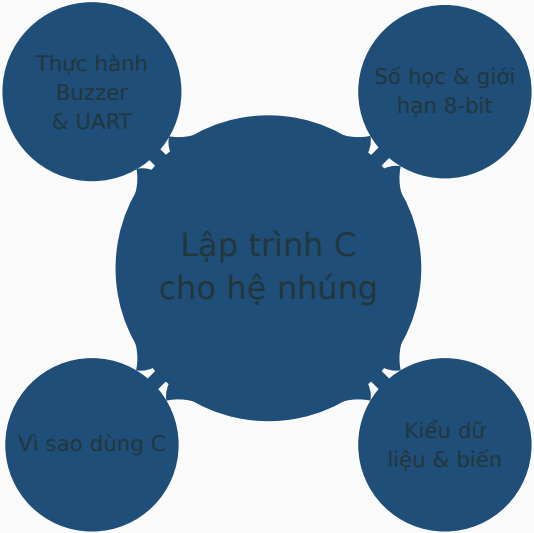
1. Vì sao phép cộng `uint32_t` trên AVR 8-bit cần 4 lệnh máy, còn phép nhân 2 số `uint16_t` cần tới 9 lệnh?
2. Một hệ thống đo nhiệt độ cần hiển thị số thập phân (vd: 25.5°C) nhưng CPU rất hạn chế tốc độ. Nên dùng `float` hay fixed-point? Vì sao?

Kiểm tra nhanh — Gợi ý đáp án

1. Cộng 32-bit chỉ cần lan truyền carry tuyến tính qua 4 byte (1 `ADD` + 3 `ADC`). Nhân 16-bit phải phân rã thành nhiều phép nhân 8×8 từng phần (vì ALU chỉ nhân được 8-bit) rồi cộng dồn các phần lại — độ phức tạp cao hơn hẳn phép cộng dù cùng là số 16-bit.
2. Nên dùng **fixed-point**: chỉ cần độ chính xác 1 chữ số thập phân (nhân 10, lưu số nguyên), tránh hoàn toàn chi phí thư viện float (có thể tốn hàng trăm byte và hàng chục–hàng trăm chu kỳ mỗi phép toán) mà vẫn đủ chính xác cho ứng dụng.

Tổng kết chương

Bản đồ tư duy — Tuần 3



Tóm tắt nguyên tắc vàng lập trình C nhúng

- Luôn dùng kiểu chính xác cố định (`stdint.h`) — chọn kiểu **nhỏ nhất đủ dùng**.
- Bắt buộc `volatile` cho thanh ghi phần cứng và biến chia sẻ với ISR.
- Dùng `PROGMEM` cho hằng số/bảng dữ liệu lớn để tiết kiệm SRAM.
- Luôn bọc ngoặc tham số và biểu thức khi viết macro `#define`.
- Tránh `float`, đệ quy sâu, và `malloc`/`free` trong mã chạy thường xuyên hoặc thời gian thực.

Câu hỏi ôn tập cuối chương

1. Kể tên 4 bước của luồng biên dịch avr-gcc và công cụ thực hiện từng bước.
2. Vì sao `int` trên AVR chỉ 16-bit chứ không phải 32-bit như trên PC?
3. Giải thích cơ chế lan truyền carry khi cộng số 32-bit trên CPU 8-bit.
4. Nêu 3 kỹ thuật/thói quen giúp tiết kiệm bộ nhớ SRAM trên vi điều khiển AVR.

Lộ trình các tuần tiếp theo

- **Tuần 1–2**: Tổng quan hệ thống nhúng; Kiến trúc và tập lệnh AVR.
- **Tuần 3** (hôm nay): Lập trình C cho hệ thống nhúng.
- **Tuần 4**: Bộ nhớ, Bus dữ liệu và Tối ưu hoá.
- **Tuần 5**: Ngắt, Timer/Counter và Watchdog.
- **Tuần 6–8**: Giao tiếp nối tiếp, RTOS, điều khiển & tổng kết.

Chuẩn đầu ra đã đạt được

CLO2 — Hoàn thành một phần

Sinh viên đã có khả năng lập trình bằng C cho vi điều khiển AVR: khai báo đúng kiểu dữ liệu, thao tác thanh ghi/bit an toàn, đánh giá chi phí tính toán, và viết chương trình điều khiển ngoại vi cơ bản (buzzer, UART).

Nội dung Tuần 4 sẽ đào sâu thêm về tổ chức bộ nhớ và kỹ thuật tối ưu hoá mã C, hoàn thiện CLO2.

Tài liệu tham khảo — Tuần 3

- Ryan Robucci, *CMPE 311 — Lecture: C Basics*, UMBC.
- Ryan Robucci, *CMPE 311 — Lecture: C Functions and Program Structure*, UMBC.
- Phillip Koopman, *18-348 — L05: Process and Design*, CMU.
- Phillip Koopman, *18-348 — L06: Embedded Language Use*, CMU.
- Phillip Koopman, *18-348 — L07: Multiprecision Arithmetic*, CMU.
- Microchip/Atmel, *ATmega328P Datasheet*; avr-libc User Manual (`avr-libc.nongnu.org`).

Thực hành — Tuần 3

Mục tiêu buổi thực hành

- Lập trình điều khiển loa Piezo (buzzer) phát âm thanh bằng kỹ thuật delay-based.
- Cấu hình và sử dụng giao tiếp UART cơ bản để gửi dữ liệu từ Arduino Uno lên máy tính.
- Hoàn thành bài Lab: gửi chuỗi `"Hello Embedded System"` qua UART, quan sát trên Serial Monitor.
- Kết hợp nhiều ngoại vi (nút nhấn, buzzer, UART) trong cùng một chương trình.

Chuẩn bị phần cứng

- Board Arduino Uno (ATmega328P).
- Loa Piezo (buzzer) thụ động hoặc chủ động.
- 1 nút nhấn (push button), 1 điện trở kéo (nếu cần), breadboard, dây nối.
- Cáp USB kết nối board với máy tính (dùng chung cho nạp chương trình và UART qua USB-Serial).

Giới thiệu loa Piezo (Buzzer)

- Loa Piezo hoạt động dựa trên **hiệu ứng áp điện** (piezoelectric effect): một màng gốm áp điện dao động cơ học khi có điện áp xoay chiều đặt vào, tạo ra sóng âm.
- **Buzzer chủ động (active)**: có mạch dao động tích hợp sẵn bên trong — chỉ cần cấp nguồn DC là kêu ở 1 tần số cố định.
- **Buzzer thụ động (passive)**: không có mạch dao động — cần vi điều khiển tạo tín hiệu dao động (tần số mong muốn) để điều khiển, linh hoạt phát nhiều cao độ khác nhau.
- Bài Lab hôm nay dùng phương án thụ động — vi điều khiển tự tạo dao động bằng phần mềm.

Sơ đồ đấu nối loa Piezo



Chân D9 (PB1) điều khiển trực tiếp loa Piezo thụ động — vi điều khiển tạo xung vuông đổi mức liên tục để sinh dao động âm thanh.

Nguyên lý tạo âm thanh bằng phần mềm

- Âm thanh là sóng dao động ở một **tần số** (Hz) nhất định — tai người nghe được khoảng 20 Hz – 20 kHz.
- Để tạo âm 1000 Hz, cần đảo mức điện áp trên chân điều khiển **1000 lần/giây**, tức chu kỳ $T = 1/1000\text{s} = 1\text{ms}$, mỗi nửa chu kỳ (bật/tắt) dài 500 μs.
- Kỹ thuật delay-based: dùng vòng lặp bật chân $\rightarrow$ trễ nửa chu kỳ $\rightarrow$ tắt chân $\rightarrow$ trễ nửa chu kỳ $\rightarrow$ lặp lại đủ số chu kỳ mong muốn.
- Tần số càng cao, thời gian trễ mỗi nửa chu kỳ càng ngắn — âm thanh càng ”cao” (the).

Code C: phát âm thanh qua buzzer

```c
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#define BUZZER_PIN PB1
void phat_am_1khz(uint16_t so_chu_ky) {   // nua chu ky 500us ~ 1kHz
    for (uint16_t i = 0; i < so_chu_ky; i++) {
        PORTB |= (1 << BUZZER_PIN);  _delay_us(500);
        PORTB &= ~(1 << BUZZER_PIN); _delay_us(500);
    }
}
int main(void) {
    DDRB |= (1 << BUZZER_PIN);
    while (1) {
        phat_am_1khz(1000);   // phat am ~1 giay
        _delay_ms(500);       // nghi ~0.5 giay
    }
}
```

Nhắc lại giao tiếp UART — khung truyền dữ liệu



- Khung truyền chuẩn ”8N1”: 1 bit Start (luôn mức 0) $\rightarrow$ 8 bit dữ liệu (LSB trước) $\rightarrow$ 1 bit Stop (luôn mức 1).
- Không đồng bộ (asynchronous): không có xung clock chung — 2 bên phải thống nhất trước **tốc độ baud** (số bit/giây).

Cấu hình tốc độ baud — thanh ghi UBRR

- Tốc độ baud được thiết lập qua thanh ghi 12-bit `UBRR0` (chia thành `UBRR0H` — 4 bit cao, `UBRR0L` — 8 bit thấp).
- Công thức tính (chế độ bất đồng bộ thường):

$$\text{UBRR} = \frac{F_{CPU}}{16 \times \text{baud}} - 1$$

- Ví dụ thực tế: $F_{CPU} = 16\text{MHz}$, baud $= 9600$ bps $\rightarrow$ $\text{UBRR} = \frac{16{,}000{,}000}{16 \times 9600} - 1 \approx 103.17 \rightarrow 103$.
- Đã kiểm chứng bằng tính toán: giá trị 103 cho tốc độ thực tế ≈ 9615 bps — sai số chỉ $\approx 0.16\%$, hoàn toàn chấp nhận được.

Các thanh ghi UART chính trên ATmega328P

Thanh ghi	Chức năng
`UBRR0H/L`	Thiết lập tốc độ baud (12-bit, chia 2 thanh ghi)
`UCSR0A`	Cờ trạng thái — bit `UDRE0` báo bộ đệm truyền đã rỗng
`UCSR0B`	Bật/tắt thu-phát — bit `TXEN0` (cho phép truyền), `RXEN0` (cho phép nhận)
`UCSR0C`	Định dạng khung truyền — số bit dữ liệu, parity, số bit stop
`UDR0`	Thanh ghi dữ liệu — ghi để truyền, đọc để nhận

Gửi ký tự qua UART bằng C

```c
#define F_CPU 16000000UL
#define BAUD  9600
#define UBRR_VALUE ((F_CPU/(16UL*BAUD))-1)
#include <avr/io.h>

void UART_init(unsigned int ubrr) {
    UBRR0H = (unsigned char)(ubrr >> 8);
    UBRR0L = (unsigned char)ubrr;
    UCSR0B = (1 << TXEN0);                          // cho phep truyen (TX)
    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);         // 8 data bit, 1 stop bit
}

void UART_transmit(unsigned char data) {
    while (!(UCSR0A & (1 << UDRE0)));    // cho bo dem truyen rong
    UDR0 = data;
}
```

Đã kiểm chứng bằng avr-gcc — biên dịch sạch, đúng chuẩn ATmega328P.

Lab: Gửi ”Hello Embedded System” qua UART — đề bài

Yêu cầu

Viết chương trình Arduino Uno khởi tạo UART ở tốc độ 9600 bps, gửi chuỗi `"Hello Embedded System"` lặp lại mỗi giây. Mở *Serial Monitor* (9600 baud) để quan sát kết quả.

- Gợi ý: viết thêm hàm `UART_print()` gửi từng ký tự của một chuỗi cho tới ký tự kết thúc `'\0'`.
- Nộp báo cáo: mã nguồn + ảnh chụp màn hình Serial Monitor.

Lab: Code mẫu gửi chuỗi qua UART

```c
void UART_print(const char *str) {
    while (*str) {
        UART_transmit(*str++);
    }
}

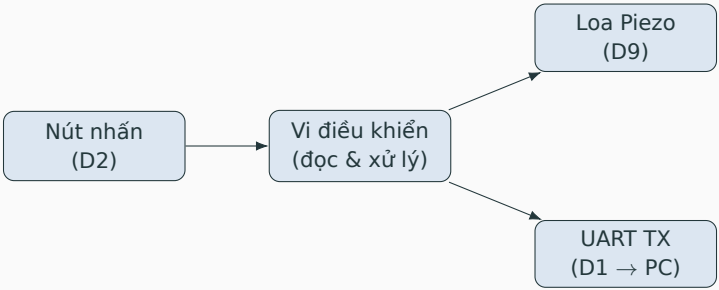
int main(void) {
    UART_init(UBRR_VALUE);
    while (1) {
        UART_print("Hello Embedded System\r\n");
        _delay_ms(1000);
    }
}
```

Đã biên dịch thành công bằng avr-gcc — chương trình đầy đủ chỉ 256 byte (232 byte Flash `.text` + 24 byte dữ liệu chuỗi).

Xem kết quả trên Serial Monitor

1. Nạp chương trình vào board qua Arduino IDE (*Upload*).
2. Mở **Tools** $\rightarrow$ **Serial Monitor**.
3. Chọn đúng tốc độ baud **9600** ở góc dưới cửa sổ Serial Monitor — nếu chọn sai, dữ liệu hiển thị sẽ là ký tự lỗi (mojibake).
4. Quan sát dòng chữ `"Hello Embedded System"` xuất hiện lặp lại mỗi giây.

Mở rộng: Nhấn nút → phát âm thanh + gửi log UART



Khi phát hiện nút nhấn chuyển trạng thái: phát 1 tiếng ”bíp” ngắn qua buzzer **đồng thời** gửi dòng log qua UART để ghi lại sự kiện — mô phỏng hệ thống cảnh báo đơn giản.

Code mẫu: kết hợp nút nhấn + buzzer + UART log

```c
#define BUTTON_PIN PD2
#define BUZZER_PIN PB1

int main(void) {
    DDRB  |= (1 << BUZZER_PIN);
    DDRD  &= ~(1 << BUTTON_PIN);     // D2 la ngo vao
    PORTD |= (1 << BUTTON_PIN);      // bat dien tro keo len noi (pull-up)
    UART_init(UBRR_VALUE);

    while (1) {
        if (!(PIND & (1 << BUTTON_PIN))) {   // nut nhan = muc THAP
            phat_am_1khz(300);
            UART_print("Nut nhan duoc bam!\r\n");
            _delay_ms(200);                  // chong doi phim (debounce)
        }
    }
}
```

Kỹ thuật debug kiểu printf qua UART

- Vi điều khiển không có màn hình — cách debug đơn giản và phổ biến nhất là **gửi giá trị biến qua UART** ra Serial Monitor, tương tự `printf()` khi lập trình trên PC.
- Kỹ thuật cơ bản: mỗi khi cần kiểm tra một biến, gọi `UART_print()` kèm nhãn mô tả (vd: `"gia_tri ADC = ..."`) và giá trị đã chuyển thành chuỗi.
- Nâng cao hơn: avr-libc hỗ trợ ánh xạ `stdio.h` (`printf()` chuẩn) vào UART qua `fdevopen()` — ngoài phạm vi buổi thực hành hôm nay, sinh viên có thể tự tìm hiểu thêm.
- Ưu điểm so với debugger phần cứng: không cần thiết bị debug chuyên dụng, chỉ cần cáp USB đã có sẵn.

Bài tập về nhà

1. Thay đổi tần số phát của buzzer thành 500 Hz và 2000 Hz — nhận xét sự khác biệt về cao độ nghe được.
2. Viết hàm `UART_print_number()` chuyển một số `uint16_t` thành chuỗi ký tự và gửi qua UART (không dùng `printf`).
3. Kết hợp cảm biến/nút nhấn để phát 2 âm khác nhau tương ứng 2 trạng thái (vd: còn hàng / hết hàng), đồng thời log trạng thái qua UART.
4. Viết lại ví dụ macro `SQUARE_BAD` ở Chương 2 thành phiên bản an toàn và giải thích bằng lời vì sao nó đúng.

Tiêu chí đánh giá buổi thực hành

Tiêu chí	Điểm
Buzzer phát đúng âm thanh theo yêu cầu	2
Lab UART gửi đúng chuỗi ”Hello Embedded System”	4
Kết hợp nút nhấn + buzzer + UART log hoạt động đúng	2
Báo cáo rõ ràng, đầy đủ	1
Hoàn thành đúng thời gian	1

Vì sao dùng C cho ARM Cortex-M

Nhắc lại: Lập trình hợp ngữ Thumb-2 (Tuần 2)

- Tuần 2 (phần ARM Cortex-M) đã lập trình trực tiếp bằng hợp ngữ Thumb-2: từng lệnh `MOVS`, `LDR`, `STR`, `BL`... ánh xạ đúng 1:1 với một lệnh máy của lõi Cortex-M.
- Ưu điểm: kiểm soát tuyệt đối phần cứng, hiệu năng tối đa, kích thước mã tối thiểu — giống hệt nhận xét đã rút ra với Assembly AVR.
- Nhược điểm cảm nhận được: chương trình dài, khó đọc lại — càng rõ hơn trên Cortex-M vì tập lệnh Thumb-2 và số thanh ghi ngoại vi nhiều hơn AVR đáng kể.
- Tuần 3: chuyển sang ngôn ngữ **C** trên nền Cortex-M — cùng triết lý như đã làm với AVR, nhưng công cụ và một số chi tiết ngôn ngữ sẽ khác.

Vì sao không lập trình thuần Assembly cho Cortex-M?

- Cùng lý do đã nêu với AVR: chương trình thực tế hàng nghìn dòng lệnh, khó bảo trì bằng Assembly.
- Cortex-M còn có **nhiều ngoại vi và thanh ghi cấu hình hơn hẳn** AVR (nhiều bus AHB/APB, nhiều chế độ clock, DMA...) — viết tay Assembly cho toàn bộ cấu hình gần như bất khả thi trong dự án thực tế.
- Assembly Thumb-2 vẫn gắn chặt với kiến trúc ARM cụ thể — đổi từ Cortex-M3 sang Cortex-M4/M7 vẫn cần hiểu sự khác biệt tập lệnh mở rộng (DSP, FPU).
- Áp lực thời gian ra thị trường càng lớn hơn với các dự án Cortex-M (thường phức tạp hơn dự án AVR) — công cụ như STM32CubeMX sinh code cấu hình càng thể hiện rõ giá trị của việc lập trình bậc cao hơn.

Ngôn ngữ C trên Cortex-M: vẫn là ”hợp ngữ bậc cao”

Vị trí của C — không đổi so với AVR

C vẫn đóng đúng vai trò ngôn ngữ bậc trung: đủ trừu tượng để viết biểu thức/hàm/cấu trúc điều khiển, nhưng vẫn đủ gần phần cứng để thao tác trực tiếp thanh ghi 32-bit, con trỏ, bộ nhớ.

- Không có máy ảo, không thu gom rác — vẫn đúng trên Cortex-M dù CPU mạnh hơn AVR nhiều lần.
- Hơn 95% firmware nhúng trên thế giới (bao gồm cả các dự án Cortex-M) được viết bằng C hoặc C/C++ — CMSIS (chuẩn của ARM) cũng được định nghĩa hoàn toàn bằng C.

C vs Assembly trên Cortex-M: Năng suất lập trình

- C cho phép biểu diễn phép toán phức tạp bằng biểu thức, giống AVR: `c = (a + b) * 2;` thay vì nhiều dòng `ADDS`/`MULS`.
- Với Cortex-M, lợi ích năng suất còn **lớn hơn AVR**: STM32CubeMX tự sinh hàng trăm dòng code khởi tạo clock/GPIO/ngoại vi mà lập trình Assembly tay gần như không thể tái tạo trong thời gian hợp lý.
- Hàm, thư viện HAL/CMSIS, cấu trúc điều khiển giúp tổ chức mã rõ ràng hơn nhiều so với chương trình Assembly Thumb-2 tương đương.

C vs Assembly trên Cortex-M: Hiệu năng và kiểm soát phần cứng

- Assembly Thumb-2 vẫn cho phép kiểm soát từng chu kỳ xung nhịp — quan trọng cho ISR cực kỳ nhạy thời gian hoặc driver tốc độ cao (tương tự AVR).
- `arm-none-eabi-gcc` với tối ưu hoá (`-Os`, `-O2`) sinh mã rất gần tối ưu Assembly viết tay cho các đoạn mã đơn giản — cùng kết luận như với avr-gcc.
- C trên Cortex-M vẫn thao tác trực tiếp thanh ghi qua địa chỉ bộ nhớ (`GPIOA->ODR`...) — không mất khả năng truy cập phần cứng so với Assembly.

Bảng so sánh tổng hợp C và Assembly trên Cortex-M

Tiêu chí	Assembly Thumb-2	C
Năng suất lập trình	Thấp — viết từng lệnh máy	Cao — biểu thức, hàm, HAL/CMSIS
Tính di động	Không — gắn chặt lõi Cortex-M cụ thể	Cao — biên dịch lại cho MCU/dòng Cortex-M khác
Dễ đọc / bảo trì	Khó, nhiều thanh ghi cấu hình	Dễ hơn nhiều nhờ HAL
Kiểm soát phần cứng	Tuyệt đối, từng chu kỳ	Gián tiếp, qua trình biên dịch
Hiệu năng tối ưu	Tối đa (nếu tay nghề cao)	Rất tốt với -0s/-02
Thời gian phát triển	Dài, đặc biệt với cấu hình clock/ngoại vi phức tạp	Ngắn hơn nhiều, có công cụ sinh code (CubeMX)

Khi nào vẫn cần Assembly trong dự án Cortex-M dùng C?

- Đoạn mã khởi động cực sớm (startup code, `Reset_Handler`) — trước khi ngăn xếp và biến `.data`/`.bss` sẵn sàng (đã trình bày chi tiết ở tài liệu tham khảo về quy trình biên dịch Cortex-M).
- ISR cực kỳ nhạy thời gian, hoặc mã cần dùng lệnh CPU đặc biệt mà C không biểu diễn trực tiếp (vd một số lệnh DSP trên Cortex-M4).
- Thực tế phổ biến: viết **đa số bằng C**, chỉ startup code và đôi khi vài dòng inline assembly (`__asm`) cho thao tác đặc biệt (vd `WFI`, thao tác thanh ghi hệ thống `MSR`/`MRS`).

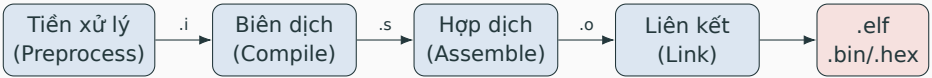
Trình biên dịch arm-none-eabi-gcc và CMSIS/HAL

- **arm-none-eabi-gcc**: phiên bản GCC được biên dịch chéo để sinh mã máy Thumb-2 cho lõi Cortex-M — vai trò tương đương avr-gcc, chạy trên PC thông thường.
- **CMSIS** (Cortex Microcontroller Software Interface Standard): chuẩn của ARM định nghĩa cách truy cập thanh ghi lõi/ngoại vi thống nhất — vai trò tương đương `avr/io.h`, nhưng là chuẩn chung cho **mọi** hãng sản xuất chip Cortex-M, không chỉ riêng một nhà cung cấp.
- **STM32 HAL**: thư viện của ST xây dựng trên nền CMSIS, cung cấp hàm bậc cao hơn (`HAL_GPIO_Init()`…) — vai trò tương đương avr-libc nhưng đầy đủ và phức tạp hơn nhiều.
- STM32CubeIDE dùng arm-none-eabi-gcc/CMSIS/HAL bên dưới — cũng giống cách Arduino IDE ẩn avr-gcc/avr-libc phía sau giao diện thân thiện.

Bộ công cụ ARM Toolchain

- `arm-none-eabi-gcc` — biên dịch mã nguồn C/C++ thành mã máy Thumb-2.
- `CMSIS + STM32 HAL` — thư viện chuẩn truy cập thanh ghi lõi/ngoại vi (vai trò tương đương avr-libc).
- `arm-none-eabi-as` — hợp dịch: sinh mã máy từ hợp ngữ Thumb-2.
- `arm-none-eabi-ld` — liên kết: gộp file object + thư viện + **linker script** thành file thực thi.
- `arm-none-eabi-objcopy` — chuyển định dạng file (`.elf` → `.bin`/`.hex`) để nạp vào chip.
- `arm-none-eabi-objdump` — dịch ngược mã máy Thumb-2 để kiểm tra.
- `ST-Link Utility / OpenOCD` — nạp file `.bin` vào Flash qua SWD, vai trò tương đương avrdude.

Luồng biên dịch: Preprocess → Compile → Assemble → Link



Bốn bước **giống hệt luồng avr-gcc** về khái niệm; khác biệt duy nhất đáng chú ý là bước Liên kết trên Cortex-M **bắt buộc** cần một linker script (`.ld`) mô tả vùng Flash/RAM — đã trình bày chi tiết ở mục 8.2 của tài liệu tham khảo Cortex-M.

Bước 1: Tiền xử lý (Preprocessing) — không đổi

- Do bộ tiền xử lý C (cpp) thực hiện — hoàn toàn giống AVR, không phụ thuộc kiến trúc CPU đích.
- Mở rộng `#include`, `#define`, xử lý `#ifdef`/`#ifndef`/`#endif`.
- Kết quả trung gian: file `.i` — mã C thuần.
- Đây là bước **duy nhất** trong 4 bước hoàn toàn giống nhau bất kể CPU đích là AVR hay Cortex-M.

Bước 2–3: Biên dịch và Hợp dịch (arm-none-eabi-gcc/-as)

- **Biên dịch**: `arm-none-eabi-gcc` phân tích cú pháp `.i`, kiểm tra kiểu, tối ưu hoá, sinh hợp ngữ **Thumb-2** tương ứng (file `.s`) — thay vì hợp ngữ AVR.
- **Hợp dịch**: `arm-none-eabi-as` dịch file `.s` thành file đối tượng `.o`.
- File `.o` vẫn chưa hoàn chỉnh: lời gọi hàm thư viện (như `HAL_Delay`) chỉ là tham chiếu chưa giải quyết — cùng cơ chế như `_delay_ms` trên AVR.

Bước 4: Liên kết (Linking) và tạo file nạp trên Cortex-M

- **Liên kết**: `arm-none-eabi-ld` kết hợp mọi file `.o` với thư viện CMSIS/HAL cần thiết, **và đọc linker script** để biết chính xác vùng Flash/RAM của chip đích (STM32F103C8: Flash 64KB tại `0x08000000`, RAM 20KB tại `0x20000000`).
- Kết quả: file `.elf` — chứa cả `.isr_vector` (bảng vector ngắt) mà AVR không có khái niệm tương đương.
- `arm-none-eabi-objcopy` chuyển `.elf` sang `.bin`/`.hex` để nạp qua ST-Link/SWD.

Lưu ý — khác biệt quan trọng với AVR

Trên AVR, linker script có sẵn ẩn trong avr-gcc, hiếm khi cần chỉnh sửa. Trên Cortex-M, linker script (`.ld`) thường **do STM32CubeMX tự sinh riêng cho từng chip**, và cần hiểu đúng nếu dự án có bootloader hoặc nhiều vùng bộ nhớ.

Cấu trúc chương trình C nhúng cơ bản (STM32 HAL)

```c
#include "stm32f1xx_hal.h"

int main(void) {
    HAL_Init();
    // Khoi tao (init): cau hinh clock, GPIO, ngoai vi 1 lan
    __HAL_RCC_GPIOC_CLK_ENABLE();
    GPIO_InitTypeDef g = {0};
    g.Pin = GPIO_PIN_13; g.Mode = GPIO_MODE_OUTPUT_PP;
    HAL_GPIO_Init(GPIOC, &g);

    while (1) {                     // Vong lap vo han (super-loop)
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
        HAL_Delay(500);
    }
    // Khong bao gio den duoc day
}
```

Cấu trúc **giống hệt** chương trình AVR: khởi tạo một lần rồi lặp vô hạn — chỉ khác tên hàm khởi tạo và hằng số cấu hình.

Hàm main() và vòng lặp vô hạn trên Cortex-M

- Mọi chương trình C vẫn bắt đầu tại `main()` — không đổi so với AVR.
- Khác biệt cốt lõi giống hệt: chương trình nhúng Cortex-M cũng **không bao giờ kết thúc**, nên `main()` luôn chứa vòng lặp `while(1)` (super-loop).
- Điểm khác biệt nhỏ: trên Cortex-M, `main()` được gọi **sau** khi `Reset_Handler` hoàn tất chuỗi bảy bước khởi tạo môi trường C (nạp SP, sao chép `.data`, xoá `.bss`...) — trên AVR, avr-libc cũng làm việc tương tự nhưng đơn giản hơn nhiều (không có bảng vector ngắt phức tạp).

Vì sao main() không return trên Cortex-M?

Vẫn không có hệ điều hành để ”quay về”

Giống hệt lý do trên AVR: vi điều khiển Cortex-M chạy bare-metal (không hệ điều hành, trừ khi cố ý chạy RTOS) không có nơi nào để ”quay về” nếu `main()` return.

- Khác biệt kỹ thuật: mã khởi động Cortex-M (`Reset_Handler`, xem tài liệu tham khảo mục 8.3) xử lý trường hợp `main()` return bằng một vòng lặp vô hạn **tường minh trong assembly** (`while (1) { }` ở cuối `Reset_Handler`) — cùng triết lý an toàn như avr-libc nhưng lộ rõ hơn trong mã nguồn startup.
- Vì vậy `int main(void)` trên Cortex-M cũng không bao giờ dùng lệnh `return`.

Đọc mã Assembly sinh từ C bằng arm-none-eabi-objdump

- Lệnh `arm-none-eabi-objdump -S chương_trình.elf` in ra mã Thumb-2 xen kẽ dòng mã C tương ứng — vai trò giống hệt `avr-objdump -S`.
- Hữu ích cho cùng mục đích: kiểm tra tối ưu hoá, kích thước mã, hoặc học cách trình biên dịch dịch một cấu trúc C cụ thể sang Thumb-2.
- Định dạng dòng tương tự: *địa chỉ: mã máy (hex) lệnh Thumb chú thích*.

Ví dụ: từ C sang Assembly — mã nguồn C

```c
#include "stm32f1xx.h"

void bat_led(void) {
    GPIOC->BSRR = (1 << 13);   // Set bit 13 an toan (atomic)
}
```

Biên dịch bằng: `arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -Os -c bat_led.c -o bat_led.o`

Ví dụ: từ C sang Assembly — kết quả disassembly

```
00000000 <bat_led>:
   0:   4a02   LDR   R2, [PC, #8]   ; nap dia chi GPIOC->BSRR
   2:   2120   MOVS  R1, #0x2000    ; hang so bit 13
   4:   6011   STR   R1, [R2]       ; ghi vao BSRR
   6:   4770   BX    LR
```

- Khác AVR (nơi `PORTB |= (1<<PB5)` dịch thành đúng **1 lệnh** `SBI`), trên Cortex-M thao tác ghi `BSRR` cần **4 lệnh**: nạp địa chỉ thanh ghi (literal pool, mục 5.2 của tài liệu tham khảo), nạp hằng số bit, ghi, rồi trở về.
- Lý do: Cortex-M không có lệnh 1-chu-kỳ thao tác trực tiếp một địa chỉ ngoại vi cố định như `SBI` của AVR — mọi truy cập bộ nhớ đều cần nạp địa chỉ vào thanh ghi trước.

So sánh hiệu năng: C tối ưu vs Assembly viết tay (Cortex-M)

Thao tác	Assembly tay	C (-0s)
Ghi bit qua BSRR (`GPIOC->BSRR = ...`)	4 lệnh	4 lệnh (giống hệt)
Đọc-sửa-ghi ODR (`GPIOC->ODR |= ...`)	5–6 lệnh	5–6 lệnh (giống hệt)
Toggle qua XOR (`GPIOC->ODR ^= ...`)	5–6 lệnh thủ công	5–6 lệnh (giống)

- Kết luận **giống hệt** AVR: với mẫu hình đơn giản, C tối ưu cho ra đúng bằng Assembly viết tay — không mất hiệu năng, dù số lệnh tuyệt đối trên Cortex-M nhiều hơn AVR cho cùng thao tác GPIO (vì không có lệnh 1-chu-kỳ chuyên dụng như `SBI`/`CBI`).
- Bù lại, mỗi lệnh Cortex-M chạy ở tần số cao hơn nhiều (72MHz so với 16MHz) nên thời gian thực tế vẫn thường nhanh hơn AVR dù số lệnh nhiều hơn.

Ví dụ đối chiếu: cùng chức năng — AVR (Tuần 2) vs Cortex-M (Tuần 3)

- Chương trình chớp tắt LED viết bằng C với `GPIOC->BSRR`, `HAL_Delay()` biên dịch (`-Os`) ra kích thước lớn hơn 162 byte của AVR — chủ yếu do bảng vector ngắt (`.isr_vector`) và mã khởi tạo clock hệ thống mà AVR không có.
- Nếu tự tay viết Assembly Thumb-2 tương đương, kích thước và tốc độ phần thân vòng lặp gần như **giống hệt** bản C — cùng kết luận sư phạm như AVR.
- **Kết luận sư phạm (không đổi so với AVR)**: C không ”trả giá” hiệu năng cho thao tác GPIO/định thời cơ bản trên Cortex-M, nhưng giúp ta không phải tự quản lý clock, ngăn xếp, hay bảng vector ngắt bằng tay.

Kiểm tra nhanh — Câu hỏi

Thảo luận

1. Vì sao `GPIOC->BSRR = (1<<13)` trên Cortex-M cần tới 4 lệnh máy, trong khi `PORTB |= (1<<PB5)` trên AVR chỉ cần 1 lệnh `SBI`?
2. Nêu đúng thứ tự 4 bước của luồng biên dịch arm-none-eabi-gcc, và điểm khác biệt duy nhất so với luồng avr-gcc.

Kiểm tra nhanh — Gợi ý đáp án

1. Vì AVR có lệnh `SBI` chuyên dụng thao tác trực tiếp 32 thanh ghi I/O đầu tiên bằng địa chỉ 5-bit nhúng ngay trong mã lệnh; Cortex-M không có lệnh tương đương — mọi thanh ghi ngoại vi đều là địa chỉ 32-bit đầy đủ trong không gian bộ nhớ phẳng, phải nạp địa chỉ vào thanh ghi (qua literal pool) trước khi ghi.
2. (1) Tiền xử lý — `cpp`; (2) Biên dịch — `arm-none-eabi-gcc` sinh hợp ngữ Thumb-2; (3) Hợp dịch — `arm-none-eabi-as` sinh `.o`; (4) Liên kết — `arm-none-eabi-ld` gộp thành `.elf` (cần thêm linker script `.ld` mà avr-gcc không yêu cầu tường minh), sau đó `arm-none-eabi-objcopy` tạo `.bin`/`.hex`.

Kiểu dữ liệu và biến trong C trên Cortex-M

Vì sao kiểu dữ liệu vẫn quan trọng trên Cortex-M?

- STM32F103C8 có **20 KB SRAM** — nhiều gấp 10 lần AVR (2KB), nhưng vẫn hữu hạn: dự án phức tạp (nhiều buffer UART/DMA, RTOS) vẫn có thể cạn bộ nhớ nếu chọn kiểu dữ liệu bất cẩn.
- CPU Cortex-M là bộ xử lý **32-bit**: khác AVR, các phép toán trên kiểu **đến 32-bit là ”miễn phí”** về mặt số lệnh (native), chỉ kiểu lớn hơn (64-bit) mới cần nhiều lệnh.
- Ý thức về kích thước biến vẫn cần thiết, nhưng **ngưỡng quan tâm dịch chuyển**: từ ”8-bit hay 16-bit” (AVR) sang ”32-bit hay 64-bit” (Cortex-M).

Kiểu dữ liệu chuẩn C trên Cortex-M 32-bit

Kiểu	Kích thước	Phạm vi giá trị
`char`	1 byte	-128 .. 127
`unsigned char`	1 byte	0 .. 255
`int`	**4 byte**	$\approx -2.1 \times 10^9$ .. 2.1×10^9
`unsigned int`	**4 byte**	0 .. $\approx 4.3 \times 10^9$
`long`	4 byte	giống `int` trên nền tảng 32-bit này
`unsigned long`	4 byte	giống `unsigned int`
`float`	4 byte	IEEE 754 đơn (single precision)
`double`	**8 byte**	IEEE 754 kép (double precision) — khác AVR!

Xác minh bằng `sizeof()` với arm-none-eabi-gcc: `int` là **32-bit** (khớp PC, khác

Kiểu có dấu và không dấu — không đổi so với AVR

- **Signed/Unsigned**: nguyên lý giống hệt AVR, chỉ khác biên giá trị vì thanh ghi rộng hơn.
- Ví dụ tràn số: `uint32_t a = 4000000000; uint32_t b = 1000000000; uint32_t c = a + b;` $\rightarrow$ tràn khỏi 2^{32}, kết quả sai nếu không lường trước — cùng bản chất lỗi như ví dụ `uint8_t` tràn ở AVR, chỉ khác ngưỡng tràn.
- Với thanh ghi ngoại vi CMSIS/HAL, quy ước dùng `uint32_t` không dấu là chuẩn mực (mọi thanh ghi Cortex-M rộng 32-bit và không có khái niệm ”âm”).

Vấn đề kích thước kiểu không cố định — vẫn tồn tại

Không portable — nhưng ít rủi ro hơn khi đích là PC/ARM

Chuẩn C vẫn chỉ đảm bảo `int` tối thiểu 16-bit. Trên Cortex-M (và hầu hết PC hiện đại), `int` = 32-bit — nên mã viết trên PC rồi build cho Cortex-M **ít rủi ro tràn số hơn** so với đem chạy trên AVR.

- Rủi ro ngược lại vẫn còn: mã viết/test trên Cortex-M rồi đem sang AVR (hoặc MCU 8/16-bit khác) vẫn có thể tràn số âm thầm.
- Kết luận không đổi: **luôn** dùng kiểu chính xác cố định thay vì dựa vào `int` ngầm định, bất kể đích là AVR hay Cortex-M.

Kiểu dữ liệu chính xác cố định — stdint.h vẫn là chuẩn mực

stdint.h — không đổi

Vẫn dùng `uint8_t`, `int16_t`, `uint32_t`… — CMSIS và STM32 HAL sử dụng `stdint.h` triệt để, giống avr-libc.

- Toàn bộ struct thanh ghi ngoại vi trong CMSIS (vd `GPIO_TypeDef`) khai báo trường bằng `uint32_t volatile` — ví dụ thực tế lớn nhất cho thấy tầm quan trọng của kiểu cố định.
- Khuyến nghị không đổi: luôn ưu tiên `stdint.h` trong mọi mã nguồn nhúng, dù đích là AVR hay Cortex-M.

Bảng đầy đủ các kiểu trong stdint.h (không đổi)

Kiểu	Kích thước	Phạm vi giá trị
`uint8_t`	1 byte	0 .. 255
`int8_t`	1 byte	-128 .. 127
`uint16_t`	2 byte	0 .. 65535
`int16_t`	2 byte	-32768 .. 32767
`uint32_t`	4 byte	0 .. 4294967295
`int32_t`	4 byte	$\approx \pm 2.1 \times 10^9$
`uint64_t` / `int64_t`	8 byte	Vẫn hiếm dùng, nhưng **khả thi hơn AVR** nhờ ALU 32-bit

Ví dụ code: khai báo biến với stdint.h trên Cortex-M

```c
#include <stdint.h>

uint8_t  so_lan_nhan_nut = 0;      // van dung kieu nho neu du dung
int32_t  nhiet_do_x100   = -50;    // vd: -0.50 do C, luu x100
uint32_t so_ms_da_troi_qua = 0;    // HAL_GetTick() tra ve kieu nay

void tang_bo_dem(void) {
    if (so_lan_nhan_nut < 255) {   // van kiem tra truoc khi tang
        so_lan_nhan_nut++;
    }
}
```

Quy ước không đổi: chọn kiểu nhỏ nhất đủ dùng — dù Cortex-M có nhiều SRAM hơn, thói quen tốt vẫn nên giữ (đặc biệt khi dùng RTOS, nơi mỗi task có ngăn xếp riêng, rất tốn SRAM nếu lãng phí).

Biến volatile trên Cortex-M — vấn đề không đổi

- Nguyên lý giống hệt AVR: trình biên dịch giả định biến chỉ đổi khi chính mã đang xét ghi vào — giả định sai với thanh ghi phần cứng hoặc biến bị ISR sửa.
- Trên Cortex-M, **mọi** thanh ghi ngoại vi trong CMSIS (`GPIOx->IDR`, `USART1->SR`...) đã được khai báo `volatile` sẵn trong file header — giống `avr/io.h` đã tự động thêm `volatile` cho các thanh ghi.
- Biến toàn cục người dùng tự khai báo (dùng chung giữa ISR và vòng lặp chính) vẫn **cần tự thêm** `volatile` — trách nhiệm này không đổi so với AVR.

Ví dụ lỗi: thiếu từ khoá volatile trên Cortex-M

```
uint8_t flag = 0;              // LOI: thieu volatile

void EXTI0_IRQHandler(void) {
    flag = 1;                  // ngat ngoai (EXTI) set co
    EXTI->PR = (1 << 0);       // xoa co ngat (bat buoc tren Cortex-M)
}

int main(void) {
    while (flag == 0) {        // cho ngat xay ra
        ;
    }
    GPIOC->BSRR = (1 << 13);   // KHONG BAO GIO chay toi day!
    while (1);
}
```

Vì sao lỗi? Bằng chứng từ arm-none-eabi-objdump (-0s)

```
main:                          ; flag KHONG volatile
   LDR   R2, [PC, #8]           ; nap dia chi bien flag
   LDR   R3, [R2]               ; doc flag DUY NHAT 1 LAN
   CMP   R3, #0
   BNE   .+4                    ; neu != 0, thoat vong lap
loop:
   B     loop                    ; neu == 0, LAP VO HAN TAI DAY!
```

Cùng bản chất lỗi như AVR, khác chi tiết mã máy

Vòng lặp B `loop` chỉ nhảy vào chính nó — CPU không bao giờ đọc lại `flag` từ SRAM, dù ISR có set `flag = 1`. Đây chính là ví dụ tương ứng, dựng lại bằng Thumb-2, cho lỗi RJMP `.-2` đã kiểm chứng trên AVR ở Tuần 2–3.

Ví dụ đúng: dùng volatile cho biến chia sẻ với ISR

```
volatile uint8_t flag = 0;     // DUNG: co volatile

void EXTI0_IRQHandler(void) {
    flag = 1;
    EXTI->PR = (1 << 0);
}

int main(void) {
    while (flag == 0) {        // luon doc lai flag tu SRAM
        ;
    }
    GPIOC->BSRR = (1 << 13);   // chay dung khi ngat xay ra
    while (1);
}
```

Mã máy tương ứng dùng LDR **lặp lại mỗi vòng** thay vì đọc 1 lần — cùng nguyên lý sửa lỗi như AVR (BREQ nhảy về lại LDS).

Quy tắc: khi nào bắt buộc dùng volatile — không đổi

- Biến ánh xạ trực tiếp thanh ghi phần cứng — CMSIS đã tự động thêm `volatile`, không cần khai báo lại.
- Biến toàn cục đọc trong vòng lặp chính, ghi trong ISR (hoặc ngược lại) — trường hợp phổ biến nhất, giống hệt AVR.
- Biến dùng trong vòng lặp chờ bận (busy-wait) chờ điều kiện phần cứng/ngắt.
- Không lạm dụng — `volatile` ngăn tối ưu hoá, làm chậm mã nếu dùng tràn lan cho biến không thực sự chia sẻ.

Từ khoá const trong C trên Cortex-M

- `const` vẫn báo cho trình biên dịch: biến chỉ đọc, không được ghi sau khởi tạo — ý nghĩa ngôn ngữ không đổi.
- Lợi ích không đổi: tối ưu hoá tốt hơn, cảnh báo/lỗi biên dịch nếu lỡ ghi vào biến chỉ đọc.
- Dùng phổ biến cho: bảng tra cứu, chuỗi thông báo, hệ số cấu hình cố định — giống hệt mục đích sử dụng trên AVR.

Khác biệt lớn: const trên Cortex-M KHÔNG cần PROGMEM

Điểm khác biệt quan trọng nhất của mục này

Trên Cortex-M, **mọi biến `const` mặc định đã nằm trong Flash** (section `.rodata`) và **không** bị sao chép vào SRAM lúc khởi động — khác hẳn AVR, nơi `const` thường vẫn chiếm SRAM trừ khi dùng `PROGMEM`.

- Lý do: dù Cortex-M cũng có kiến trúc Harvard biến thể (I-bus/D-bus tách biệt vật lý, mục 5 tài liệu tham khảo), **cả hai bus đều nhìn thấy cùng một không gian địa chỉ 4GB thống nhất** — CPU có thể đọc trực tiếp Flash qua D-bus mà không cần cơ chế đặc biệt như `pgm_read_byte()`.
- Đây là lợi ích thực dụng lớn của Cortex-M so với AVR khi làm việc với bảng dữ liệu/hằng số lớn.

Ví dụ code: bảng tra cứu const trên Cortex-M — đơn giản hơn AVR

```c
#include <stdint.h>

// Bang nam trong Flash (.rodata), KHONG chiem SRAM
// KHONG can PROGMEM, KHONG can pgm_read_byte()!
const uint8_t bang_led[] = {
    0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80
};

uint8_t doc_gia_tri(uint8_t i) {
    return bang_led[i];   // doc TRUC TIEP, giong doc SRAM binh thuong
}
```

So với đoạn code PROGMEM tương đương trên AVR (mục 2.9, Tuần 3 AVR): ngắn gọn hơn, không cần header `avr/pgmspace.h` hay hàm đọc chuyên dụng — đánh đổi bằng việc CPU đọc Flash chậm hơn SRAM một chút (nhưng không cần API đặc biệt).

Toán tử bit-wise trên Cortex-M — không đổi

Toán tử	Ký hiệu C	Ý nghĩa	Bảng chân trị (1 bit)
AND	&	Cả 2 bit đều 1	$1\&1=1$; còn lại $=0$
OR	|	Ít nhất 1 bit là 1	$0\|0=0$; còn lại $=1$
XOR	^	Hai bit khác nhau	$1\oplus0=1$; $1\oplus1=0$
NOT	~	Đảo ngược từng bit	$\sim1=0$; $\sim0=1$

Bốn phép toán logic này vẫn được ALU của Cortex-M thực hiện trực tiếp bằng 1 lệnh máy (`ANDS`, `ORRS`, `EORS`, `MVN`) mỗi loại — chỉ khác là toán hạng rộng 32-bit thay vì 8-bit.

Toán tử dịch bit trên Cortex-M — không đổi về ý nghĩa

- `x << n` / `x >> n`: ý nghĩa và công dụng giống hệt AVR, chỉ khác phạm vi n (0–31 cho kiểu 32-bit thay vì 0–7 cho kiểu 8-bit).
- Mask 1 bit `(1 << PB5)` kiểu AVR trở thành `(1 << 13)` (cho PC13) kiểu Cortex-M — cùng kỹ thuật, chỉ khác số bit khả dụng (32 thay vì 8).
- Dịch bit vẫn rẻ hơn nhân/chia — nhưng trên Cortex-M chênh lệch chi phí **nhỏ hơn nhiều** so với AVR, vì Cortex-M3 đã có bộ nhân/chia phần cứng (phân tích chi tiết ở mục 3).

Kỹ thuật SET một bit trên Cortex-M

```
// SET bit "bit" cua thanh ghi "reg" ve 1, giu nguyen cac bit khac
#define SET_BIT(reg, bit)   ((reg) |= (1 << (bit)))

// Vi du (dung ODR, khong atomic):
SET_BIT(GPIOC->ODR, 13);   // <=> GPIOC->ODR |= (1 << 13);

// Cach lam DUNG hon tren Cortex-M (atomic, dung BSRR):
#define SET_BIT_SAFE(port, bit) ((port)->BSRR = (1 << (bit)))
SET_BIT_SAFE(GPIOC, 13);
```

- Macro SET_BIT thông thường vẫn hoạt động đúng như trên AVR, nhưng dịch ra **nhiều lệnh hơn** (đọc-sửa-ghi ODR) và **không an toàn** khi có ngắt xen ngang.
- Macro SET_BIT_SAFE tận dụng BSRR — đặc quyền riêng của Cortex-M mà AVR không có tương đương.

Kỹ thuật CLEAR một bit trên Cortex-M

```
// CLEAR bit "bit" cua thanh ghi "reg" ve 0, giu nguyen cac bit khac
#define CLEAR_BIT(reg, bit) ((reg) &= ~(1 << (bit)))
CLEAR_BIT(GPIOC->ODR, 13);   // <=> GPIOC->ODR &= ~(1 << 13);

// Cach lam an toan hon, dung BSRR (nua cao = reset):
#define CLEAR_BIT_SAFE(port, bit) ((port)->BSRR = (1 << ((bit)+16)))
CLEAR_BIT_SAFE(GPIOC, 13);
```

- Cơ chế `~(1 << bit)` và phép `AND` giống hệt AVR về mặt toán học.
- Trình biên dịch AVR dịch mẫu hình này thành 1 lệnh `CBI`; trên Cortex-M, nếu dùng `ODR` trực tiếp sẽ ra nhiều lệnh hơn — nên ưu tiên `BSRR` khi có thể.

Kỹ thuật TOGGLE và kiểm tra một bit trên Cortex-M

```
// TOGGLE (dao) bit "bit"
#define TOGGLE_BIT(reg, bit) ((reg) ^= (1 << (bit)))

// KIEM TRA bit "bit" co dang la 1 hay khong
#define CHECK_BIT(reg, bit)  ((reg) & (1 << (bit)))

if (CHECK_BIT(GPIOA->IDR, 0)) {     // neu nut nhan dang o muc cao
    TOGGLE_BIT(GPIOC->ODR, 13);     // dao trang thai LED
}
// Hoac dung ham co san, ro rang hon:
// HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
```

Không có mẹo phần cứng ”ghi 1 vào thanh ghi đọc để toggle” như `PINx` của AVR — Cortex-M không có thanh ghi đọc-để-đảo tương đương; `HAL_GPIO_TogglePin()` bên trong thực chất vẫn đọc-sửa-ghi hoặc dùng `ODR` XOR.

Toán tử gán phức hợp — không đổi

```
uint32_t reg = 0;

reg |= (1 << 3);   // tuong duong: reg = reg | (1 << 3);
reg &= ~(1 << 3);  // tuong duong: reg = reg & ~(1 << 3);
reg ^= (1 << 3);   // tuong duong: reg = reg ^ (1 << 3);
reg <<= 2;         // tuong duong: reg = reg << 2;
reg >>= 1;         // tuong duong: reg = reg >> 1;
```

- Cú pháp op= không đổi — hoạt động y hệt trên mọi kiểu số nguyên bất kể kiến trúc CPU đích.
- Đây chính là cú pháp thực tế trong `GPIOC->CRH &= ~(0xF << 20);` mà ta đã dùng ở Tuần 2 (phần Cortex-M).

Macro tiền xử lý #define trên Cortex-M — khái niệm không đổi

- `#define` vẫn là phép thay thế văn bản thuần tuý, xử lý trước khi trình biên dịch thấy mã nguồn — hoàn toàn độc lập với kiến trúc CPU đích.
- CMSIS và STM32 HAL dùng macro **rất nhiều**: tên thanh ghi, mask bit, hằng số cấu hình (vd `GPIO_PIN_13`, `RCC_APB2ENR_IOPCEN`) đều là macro.
- Không tốn bộ nhớ chạy, không kiểu dữ liệu, không phạm vi — giống hệt AVR.

So sánh #define và hàm (function) trên Cortex-M

Tiêu chí	Macro (#define)	Hàm (function)
Chi phí gọi (overhead)	Không	Có — BL/BX LR, có thể PUSH/POP thanh ghi
Kiểm tra kiểu dữ liệu	Không	Có
Dễ debug	Khó	Dễ — breakpoint qua ST-Link
Rủi ro tác dụng phụ	Cao nếu viết sai	Thấp

Bảng giống hệt AVR về nội dung; chỉ khác cơ chế gọi hàm phần cứng (BL/LR thay vì CALL/ngăn xếp) đã học ở Tuần 2.

Macro có tham số (function-like macro) — không đổi

```
#define MAX(a, b)  ((a) > (b) ? (a) : (b))
#define MIN(a, b)  ((a) < (b) ? (a) : (b))

uint32_t x = MAX(5, 10);
uint32_t y = MIN(gia_tri_adc, 4095);   // vd gioi han theo do phan giai ADC 12-bit
```

Cú pháp và hành vi giống hệt AVR — hoàn toàn không phụ thuộc kiến trúc CPU đích.

Lỗi thường gặp: thiếu ngoặc bảo vệ trong macro — vẫn xảy ra

```
#define SQUARE_BAD(x)  x*x        // LOI: thieu ngoac

int a = 25 / SQUARE_BAD(2+3);
// Mo rong: 25 / 2+3*2+3 = 12 + 6 + 3 = 21   <-- SAI, giong het AVR!
```

Lỗi ngôn ngữ, không phải lỗi kiến trúc

Đây là lỗi ở **cấp độ tiền xử lý C**, xảy ra **giống hệt** trên AVR lẫn Cortex-M vì bộ tiền xử lý hoàn toàn không biết CPU đích là gì — bằng chứng rõ ràng cho thấy quy tắc viết macro an toàn là kiến thức **phổ quát**, không riêng một kiến trúc nào.

Cách viết macro an toàn — quy tắc không đổi

```
#define SQUARE_GOOD(x)  ((x) * (x))    // DUNG: co ngoac day du

int b = 25 / SQUARE_GOOD(2+3);
// Mo rong: 25 / ((2+3)*(2+3)) = 25/25 = 1  <-- DUNG
```

Quy tắc vàng viết macro — không đổi

1. Luôn bọc từng tham số trong ngoặc đơn.
2. Luôn bọc toàn bộ biểu thức kết quả trong ngoặc đơn.
3. Cân nhắc dùng hàm `inline` khi cần an toàn kiểu dữ liệu hơn.

Hàm trong C trên Cortex-M: khai báo và định nghĩa

```c
// Khai bao (prototype) - thuong dat trong file .h
uint16_t doc_adc(uint8_t kenh);

// Dinh nghia (definition) - than ham, dat trong file .c
uint16_t doc_adc(uint8_t kenh) {
    // ... doc gia tri ADC tu kenh chi dinh (HAL_ADC_...) ...
    return gia_tri;
}
```

Cú pháp và khái niệm khai báo/định nghĩa **không đổi** so với AVR — hoàn toàn thuộc về ngôn ngữ C, không phụ thuộc CPU đích.

Truyền tham số: by value vs by pointer — không đổi

```c
void tang_gia_tri_value(uint32_t x) {
    x = x + 1;              // chi sua BAN SAO cuc bo
}

void tang_gia_tri_pointer(uint32_t *x) {
    *x = *x + 1;             // sua truc tiep qua DIA CHI
}

uint32_t bien = 5;
tang_gia_tri_value(bien);    // bien van la 5
tang_gia_tri_pointer(&bien); // bien tro thanh 6
```

Nguyên lý giống hệt AVR; điểm khác kỹ thuật duy nhất: con trỏ trên Cortex-M rộng 32-bit (bằng đúng độ rộng thanh ghi), trong khi con trỏ trên AVR chỉ cần 16-bit (đủ địa chỉ hoá 2KB–32KB bộ nhớ).

Con trỏ cơ bản và truy cập thanh ghi phần cứng trên Cortex-M

```
// CMSIS dinh nghia thanh ghi theo dung co che nay (don gian hoa):
typedef struct {
    volatile uint32_t CRL, CRH, IDR, ODR, BSRR, BRR, LCKR;
} GPIO_TypeDef;
#define GPIOC ((GPIO_TypeDef *)0x40011000UL)

// GPIOC thuc chat la MOT CON TRO toi dia chi co dinh
GPIOC->ODR = 0xFFFF;
```

- Con trỏ vẫn lưu địa chỉ của ô nhớ — không đổi về khái niệm.
- Khác biệt thú vị: AVR ánh xạ **từng thanh ghi** bằng 1 macro con trỏ riêng lẻ (`#define PORTB (*(volatile uint8_t*)0x25)`); CMSIS trên Cortex-M ánh xạ **cả một khối ngoại vi** bằng 1 `struct`, rồi ép kiểu con trỏ struct tới địa chỉ gốc — gọn hơn khi ngoại vi có nhiều thanh ghi liên quan.

Biên dịch riêng biệt: tổ chức file .c / .h — không đổi

- Nguyên lý chia nhiều file `.c`/`.h`, `extern`, header guard — **hoàn toàn giống AVR**, thuộc về tổ chức mã nguồn C, không phụ thuộc CPU đích.
- Khác biệt thực tế: dự án STM32CubeIDE tự sinh sẵn cấu trúc file chuẩn (`Core/Inc`, `Core/Src`, `Drivers/`…) — có tổ chức rõ ràng hơn ngay từ đầu so với 1 file `.ino` đơn giản của Arduino IDE.

Ví dụ: biên dịch riêng biệt trên Cortex-M

```
// -- sensor.h --
#ifndef SENSOR_H
#define SENSOR_H
#include <stdint.h>
uint16_t doc_cam_bien(uint8_t kenh);
#endif
// -- sensor.c --
#include "sensor.h"
uint16_t doc_cam_bien(uint8_t kenh) { return 0; }
// -- main.c --
#include "sensor.h"
int main(void) {
    uint16_t v = doc_cam_bien(0);
    while (1) {}
}
```

Cấu trúc và cú pháp **giống hệt** ví dụ AVR — chỉ khác nội dung thân hàm sẽ gọi HAL/thanh ghi Cortex-M thay vì AVR.

Struct trên Cortex-M — CMSIS là ví dụ thực tế lớn nhất

```c
typedef struct {
    uint16_t gia_tri_adc;
    uint8_t  kenh;
    uint8_t  hop_le;
} CamBien_t;

CamBien_t cb1 = {0, 0, 0};

void cap_nhat(CamBien_t *cb, uint16_t gt) {
    cb->gia_tri_adc = gt;
    cb->hop_le = 1;
}
```

Cú pháp `struct` không đổi; nhưng trên Cortex-M, sinh viên còn gặp `struct` ở quy mô lớn hơn nhiều: **toàn bộ CMSIS** (`GPIO_TypeDef`, `USART_TypeDef`…) là các `struct` mô tả khối ngoại vi thật trên chip.

Phạm vi biến (scope): global, local, static — không đổi

- **Biến toàn cục**: nằm ở `.data`/`.bss` trong SRAM — khái niệm giống hệt AVR, chỉ khác vùng SRAM lớn hơn (20KB thay vì 2KB).
- **Biến cục bộ**: trên ngăn xếp — lưu ý Cortex-M có **hai** ngăn xếp tiềm năng (MSP/PSP, đã học Tuần 2) nếu dự án dùng RTOS; chương trình bare-metal đơn giản chỉ dùng MSP, giống hệt cơ chế 1-ngăn-xếp của AVR.
- **Biến `static` cục bộ**: giữ nguyên giá trị giữa các lần gọi — không đổi.
- Nguyên tắc hạn chế biến toàn cục vẫn không đổi, và còn **quan trọng hơn** khi dùng RTOS (biến toàn cục dùng chung giữa nhiều task cần đồng bộ hoá cẩn thận).

Ép kiểu (type casting) và nguy cơ mất dữ liệu — vẫn tồn tại

```
uint32_t lon = 70000;
uint16_t nho = (uint16_t)lon;   // ep kieu tuong minh: mat du lieu!
// 70000 mod 65536 = 4464 (khac AVR: nguong tran o 32-bit, khong phai 8-bit)

int32_t  x = -1;
uint32_t y = 1;
if (x < y) { /* MONG DOI: vao day vi -1 < 1 */ }
else       { /* THUC TE: x bi ep NGAM DINH thanh unsigned (4294967295) */ }
// Van SAI theo cung co che nhu AVR -> chuong trinh vao nhanh else!
```

Cùng cạm bẫy như AVR, chỉ khác ngưỡng tràn

Quy tắc ép kiểu ngầm định khi so sánh signed/unsigned là quy tắc của **chuẩn C**, áp dụng giống hệt trên mọi kiến trúc — chỉ khác giá trị tràn cụ thể ($2^{32}-1$ thay vì $2^{16}-1$).

Kiểm tra nhanh — Câu hỏi

Thảo luận

1. Vì sao bảng tra cứu `const` trên Cortex-M không cần `PROGMEM`/`pgm_read_byte()` như trên AVR?
2. Macro `#define DOUBLE(x) x+x` có an toàn trên Cortex-M không? Kết quả `DOUBLE(3+4)` là bao nhiêu?

Kiểm tra nhanh — Gợi ý đáp án

1. Vì cả I-bus và D-bus của Cortex-M cùng nhìn thấy một không gian địa chỉ 4GB thống nhất — CPU có thể đọc trực tiếp Flash qua đường D-bus bằng lệnh `LDR` thông thường, không cần cơ chế đọc-qua-hàm-chuyên-dụng như AVR (nơi CPU chỉ đọc dữ liệu được từ SRAM bằng lệnh thường, còn đọc Flash cần lệnh `LPM` riêng).
2. Không an toàn — cùng lỗi thiếu ngoặc như trên AVR, vì đây là lỗi ở cấp tiền xử lý C, độc lập kiến trúc. `DOUBLE(3+4)` vẫn mở rộng thành `3+4+3+4=14`, nhưng lộ rõ sai khi dùng trong biểu thức lớn hơn (vd `10/DOUBLE(3+4)`), giống hệt phân tích đã làm với AVR.

Tổng kết: Kiểu dữ liệu và biến trong C trên Cortex-M

- Luôn ưu tiên `stdint.h` — không đổi; nhưng `int` nay là 32-bit (khớp PC), khác AVR (16-bit).
- `volatile` vẫn bắt buộc cho biến chia sẻ với ISR; nhưng `const` **không còn cần PROGMEM** — khác biệt lớn nhất của mục này.
- Toán tử bit-wise và macro tiền xử lý hoạt động giống hệt AVR — đây là phần ”phổ quát” của ngôn ngữ C.
- Cortex-M có thêm BSRR cho thao tác bit GPIO an toàn (atomic) mà AVR không có tương đương.
- CMSIS dùng `struct` để ánh xạ cả khối ngoại vi, thay vì từng thanh ghi rời như AVR.

Số học và đặc điểm của C trên Cortex-M 32-bit

ALU 32-bit của Cortex-M: ”giới hạn” AVR không còn nữa

- ALU của Cortex-M3 xử lý **tối đa 32 bit trong 1 lệnh** — đúng bằng độ rộng của `int`/`uint32_t` chuẩn C.
- Hệ quả quan trọng: phép cộng/trừ/AND/OR/XOR trên `uint32_t` **không còn cần đa độ chính xác** — mỗi phép toán chỉ 1 lệnh máy, khác hẳn AVR (cần 4 lệnh cho cùng phép cộng 32-bit).
- ”Giới hạn 8-bit” của AVR ở mục này của Tuần 3 (AVR) gần như **biến mất** với Cortex-M — chỉ còn xuất hiện lại nếu làm việc với kiểu `uint64_t`.

Số học đa độ chính xác trên Cortex-M — vẫn cần cho 64-bit

Multi-precision arithmetic — vẫn tồn tại, chỉ dịch ngưỡng

Cùng khái niệm đã học ở AVR: khi độ rộng số **vượt quá** độ rộng ALU, phải ghép nhiều đơn vị nhỏ hơn lại — trên Cortex-M, ngưỡng này là **64-bit** (thay vì 16/32-bit như trên AVR 8-bit).

- CPU 32-bit + số 64-bit $\rightarrow$ lưu thành 2 từ (word) 32-bit, xử lý tuần tự kèm theo dõi carry giữa 2 từ — đúng nguyên lý carry propagation đã học, chỉ khác đơn vị (word 32-bit thay vì byte 8-bit).

Ví dụ Thumb-2: cộng hai số 64-bit trên Cortex-M

```
; Cong so 64-bit: {R1:R0} = {R1:R0} + {R3:R2}
ADDS  R0, R0, R2      ; cong tu THAP, cap nhat co Carry
ADCS  R1, R1, R3      ; cong tu CAO, CONG THEM co Carry tu buoc truoc
```

Đã kiểm chứng nguyên lý: hàm C `uint64_t add64(uint64_t a, uint64_t b) { return a+b; }` biên dịch (`-Os`) ra đúng 2 lệnh `ADDS`+`ADCS` — hoàn toàn tương ứng với cặp `ADD`+`ADC` của AVR khi cộng số 16-bit, chỉ khác đơn vị xử lý là 32-bit thay vì 8-bit.

Cộng/trừ 32-bit là phép toán NATIVE trên Cortex-M

- Điểm tương phản rõ nhất với AVR: cộng 2 số `uint32_t` trên AVR cần **4 lệnh** (1 `ADD` + 3 `ADC`, đã đo ở Tuần 3 AVR); trên Cortex-M chỉ cần **đúng 1 lệnh** `ADDS`.
- Tương tự cho phép trừ (`SUBS`, 1 lệnh) và các phép logic (`ANDS`/`ORRS`/`EORS`, đều 1 lệnh cho toán hạng 32-bit).
- Đây là lợi ích cụ thể, đo lường được của kiến trúc 32-bit — không chỉ là con số tiếp thị ”mạnh hơn” chung chung.

C tự động hoá số học đa độ chính xác — vẫn đúng, ở ngưỡng 64-bit

Giá trị của C không đổi, chỉ dịch ngưỡng áp dụng

Người lập trình C viết `c = a + b;` với `a, b, c` kiểu `uint64_t` — trình biên dịch tự sinh đúng cặp `ADDS`+`ADCS`, không cần tự theo dõi carry thủ công — đúng bài học đã rút ra ở AVR, chỉ khác ngưỡng kiểu dữ liệu kích hoạt cơ chế này (64-bit thay vì 16/32-bit).

- Với `uint32_t` trở xuống, C không còn ”làm hộ” việc gì đặc biệt nữa — vì phần cứng đã lo trọn trong 1 lệnh.

Phép nhân số nguyên trên Cortex-M — mạnh hơn AVR đáng kể

- Cortex-M3 có bộ nhân phần cứng **32×32-bit → 32-bit, hoàn tất trong 1 chu kỳ** (`MULS`) — so với bộ nhân 8×8-bit của AVR.
- Hệ quả: `uint32_t mul32(uint32_t a, uint32_t b) { return a*b; }` biên dịch ra **đúng 1 lệnh** `MULS` — so với AVR cần **hàng chục lệnh** cho phép nhân 32-bit (ghép nhiều phép nhân 8-bit từng phần).
- Đây là một trong những khác biệt hiệu năng lớn nhất giữa hai kiến trúc cho khối lượng tính toán vừa phải.

Chi phí phép nhân 64-bit trên Cortex-M — vẫn cần nhiều lệnh

- Nhân 2 số 64-bit **vượt quá** khả năng 1 lệnh `MULS` (chỉ hỗ trợ $32 \times 32 \rightarrow 32$, không giữ phần tràn 64-bit đầy đủ nếu không dùng biến thể `UMULL`/`SMULL`).
- Cortex-M3 có lệnh `UMULL`/`SMULL` ($32 \times 32 \rightarrow 64$-bit, 1 lệnh) hỗ trợ tốt hơn nhiều so với việc AVR phải ghép nhiều phép nhân 8-bit cho cùng độ rộng kết quả.
- Kết luận: dù vẫn tốn nhiều lệnh hơn phép nhân 32-bit, chi phí nhân số lớn trên Cortex-M vẫn **thấp hơn nhiều bậc** so với AVR cho cùng độ rộng dữ liệu.

Phép chia số nguyên trên Cortex-M — khác biệt lớn nhất với AVR

Cortex-M3 CÓ lệnh chia phần cứng!

Khác hẳn AVR (không có lệnh chia phần cứng ở bất kỳ độ rộng nào), Cortex-M3 trở lên có lệnh `UDIV`/`SDIV` (chia không dấu/có dấu, 32-bit, thường 2–12 chu kỳ tuỳ chip) thực hiện **trực tiếp bằng phần cứng**.

- `uint32_t div32(uint32_t a, uint32_t b) { return a/b; }` biên dịch thành **1 lệnh** `UDIV` trên Cortex-M3 — so với AVR luôn cần gọi hàm thư viện `__udivmodqi4`/tương đương với vòng lặp dịch-trừ 8–9 lần lặp.
- Lưu ý: Cortex-M0/M0+ (dòng thấp nhất) **không có** `UDIV`/`SDIV` — vẫn phải gọi hàm thư viện như AVR. Đây là lý do cần biết chính xác lõi Cortex-M nào đang dùng khi đánh giá chi phí phép chia.

Bảng so sánh chi phí các phép toán số học nguyên: AVR vs Cortex-M3

Phép toán (32-bit)	AVR (8-bit)	Cortex-M3 (32-bit)	Chênh lệch
Cộng/trừ	4 lệnh	**1 lệnh**	4×
Nhân	Hàng chục lệnh	**1 lệnh** (`MULS`)	Hàng chục lần
Chia	Gọi hàm thư viện (đắt)	**1 lệnh** (`UDIV`, chỉ M3+)	Rất lớn

Đây là bảng số liệu quan trọng nhất của cả tài liệu Tuần 3 để giải thích vì sao các thuật toán nặng tính toán (lọc số, xử lý tín hiệu) gần như luôn chọn Cortex-M thay vì AVR.

Quy tắc tối ưu dịch bit vẫn hữu ích, nhưng ít cấp bách hơn

- Với số chia/nhân luỹ thừa 2, dịch bit (`x >> 2` thay `x / 4`) vẫn nhanh hơn — nguyên lý không đổi.
- Khác biệt so với AVR: vì Cortex-M3 đã có `UDIV` 1 lệnh, **chênh lệch tuyệt đối** giữa dịch bit và chia thông thường (khi số chia không phải luỹ thừa 2) **nhỏ hơn nhiều** so với AVR — không còn là tối ưu ”bắt buộc” mà chỉ là tối ưu ”tốt nên có”.
- arm-none-eabi-gcc vẫn tự động tối ưu `x/4` thành dịch bit khi số chia là hằng luỹ thừa 2 — giống hành vi của avr-gcc.

Số dấu chấm động (float) trên Cortex-M3 — vẫn không có FPU

STM32F103 (Cortex-M3) vẫn không có FPU phần cứng

Giống AVR, dòng Cortex-M3 (bao gồm STM32F103) **không có bộ xử lý dấu chấm động phần cứng** — mọi phép toán `float` vẫn cần thư viện phần mềm.

- Khác biệt quan trọng cần biết: các dòng **Cortex-M4F và Cortex-M7** (không phải M3) **có FPU phần cứng** — nếu đổi board sang STM32F4/F7, phép toán `float` có thể chỉ tốn 1 vài chu kỳ thay vì hàng trăm.
- Với STM32F103 (board dùng trong học phần), kết luận về chi phí float **vẫn giống AVR**: nên tránh trong mã chạy thường xuyên.

Chi phí thư viện float phần mềm trên Cortex-M3

- Cũng như AVR, phép cộng `float` gọi hàm thư viện phần mềm (`__aeabi_fadd` trên arm-none-eabi-gcc) — không phải 1 lệnh máy.
- Vì ALU đã là 32-bit (đúng độ rộng `float`), chi phí **tương đối** thấp hơn AVR (không cần multi-precision cho riêng phần định trị 24-bit của số thực) — nhưng vẫn đắt hơn nhiều so với phép toán nguyên 32-bit tương ứng (hàng chục chu kỳ so với 1 chu kỳ).
- Kết luận thực dụng không đổi: dùng `float` có chi phí, cần cân nhắc — chỉ khác mức độ đắt đỏ (ít nghiêm trọng hơn AVR, nhưng vẫn đáng kể so với số nguyên).

Hệ quả: khi nào nên/không nên dùng float trên Cortex-M

- **Tránh** trên Cortex-M3 (không FPU): vòng lặp tần suất cao, ISR nhạy thời gian — cùng khuyến nghị như AVR.
- **Có thể chấp nhận thoải mái hơn AVR**: tính toán không quá thường xuyên, vì ALU 32-bit khiến chi phí tương đối thấp hơn.
- **Nếu dự án cho phép đổi chip**: chuyển sang Cortex-M4F/M7 (có FPU) loại bỏ gần như hoàn toàn vấn đề chi phí float — lựa chọn không tồn tại trên AVR.
- Fixed-point vẫn là giải pháp thay thế phổ biến khi cần hiệu năng tối đa hoặc dùng chip không FPU.

Fixed-point Arithmetic trên Cortex-M — cùng khái niệm Q-format

Fixed-point — không đổi, nhưng dải giá trị rộng hơn nhiều

Cùng khái niệm Qm.n đã học ở AVR: giá trị thực = số nguyên lưu trữ $\div 2^n$.

- Với ALU 32-bit, Cortex-M dùng thoải mái **Q16.16** (dùng `int32_t`: 16 bit nguyên + 16 bit thập phân) — độ chính xác và dải giá trị lớn hơn nhiều so với Q8.8 (`int16_t`) thường dùng trên AVR.
- Phép nhân fixed-point trên Cortex-M còn được lợi thêm từ `UMULL` (nhân $32\times32\rightarrow64$-bit 1 lệnh) để tránh mất độ chính xác khi dịch phải sau nhân.

Ví dụ Q-format trên Cortex-M (Q16.16)

```
typedef int32_t fixed_t;                  // Q16.16
#define FIX_SHIFT      16
#define TO_FIXED(x)    ((fixed_t)((x) * 65536))
#define FIXED_MUL(a,b) ((fixed_t)(((int64_t)(a) * (b)) >> FIX_SHIFT))

fixed_t gia_tri = TO_FIXED(3.5);                    // 3.5 -> 229376
fixed_t ket_qua = gia_tri + TO_FIXED(2.0);          // cong: 1 lenh ADDS!
fixed_t tich    = FIXED_MUL(gia_tri, TO_FIXED(2));  // 3.5*2 = 7.0
```

So với ví dụ Q8.8 trên AVR: cùng kỹ thuật, nhưng phép cộng ở đây chỉ 1 lệnh ADDS (thay vì ADC 16-bit của AVR) nhờ ALU 32-bit xử lý trọn trong 1 bước.

Vì sao vẫn nên tránh đệ quy sâu trên Cortex-M

- Mỗi lời gọi đệ quy vẫn cần `PUSH` thanh ghi + `LR` vào ngăn xếp — chi phí về nguyên lý giống hệt AVR, chỉ khác số byte mỗi lần (4 byte/thanh ghi 32-bit thay vì 1 byte/thanh ghi 8-bit của AVR — nên mỗi lần gọi đệ quy trên Cortex-M thực ra tốn **nhiều SRAM hơn theo byte tuyệt đối**, dù tỷ lệ so với tổng SRAM lại nhỏ hơn nhiều nhờ SRAM 20KB).
- SRAM 20KB vẫn hữu hạn — đệ quy sâu không kiểm soát vẫn có thể gây tràn ngăn xếp (stack overflow), đặc biệt nguy hiểm hơn nếu dùng RTOS (mỗi task có ngăn xếp riêng, nhỏ hơn SRAM tổng).
- Kết luận không đổi: ưu tiên vòng lặp, chỉ dùng đệ quy khi độ sâu được giới hạn chắc chắn.

Ví dụ: chuyển đệ quy thành vòng lặp trên Cortex-M

```
// DE QUY - ton PUSH/POP + BL/LR moi lan goi
uint32_t giai_thua_dequy(uint8_t n) {
    if (n <= 1) return 1;
    return n * giai_thua_dequy(n - 1);
}
// LAP - khong ton chi phi ngan xep, va tan dung MULS 1 chu ky
uint32_t giai_thua_lap(uint8_t n) {
    uint32_t ket_qua = 1;
    for (uint8_t i = 2; i <= n; i++) {
        ket_qua *= i;    // MULS 1 lenh, khong nhu 9-lenh tren AVR!
    }
    return ket_qua;
}
```

Quy tắc thực hành không đổi: ưu tiên vòng lặp — trên Cortex-M còn có thêm lợi ích phụ là phép nhân trong vòng lặp cực rẻ nhờ `MULS`.

Vì sao vẫn tránh malloc/free trên Cortex-M

- Ba lý do gốc **không đổi** so với AVR: phân mảnh bộ nhớ, không xác định được thời gian thực thi, rủi ro rò rỉ bộ nhớ.
- SRAM 20KB rộng rãi hơn AVR (2KB) nhưng **vẫn hữu hạn** — dự án dùng nhiều buffer DMA/UART/RTOS có thể cạn SRAM nhanh hơn tưởng tượng.
- Nguyên tắc ngành công nghiệp (ô tô, y tế, hàng không) **không đổi**: cấm `malloc()`/`free()` sau khởi tạo, bất kể kiến trúc CPU.

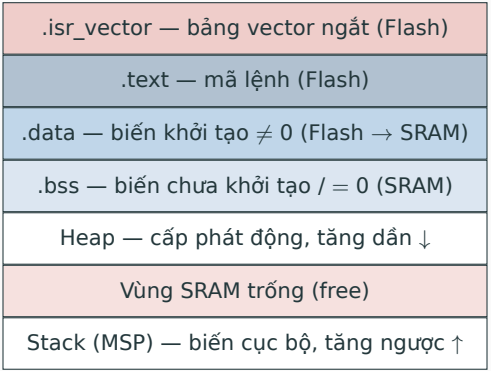
Giải pháp thay thế cấp phát động — không đổi

- **Cấp phát tĩnh**: khai báo mảng/biến kích thước cố định lúc biên dịch — nguyên lý giống hệt AVR.
- **Memory pool**: cấp trước 1 mảng lớn, tự quản lý khối cố định — giống hệt AVR.
- Trên Cortex-M khi dùng RTOS (FreeRTOS…): các API cấp phát tĩnh cho task/queue/semaphore (`xTaskCreateStatic`…) là lựa chọn được khuyến nghị thay vì bản động — mở rộng tự nhiên của nguyên tắc ”tránh cấp phát động” sang thế giới có hệ điều hành thời gian thực.

Tổ chức bộ nhớ chương trình C trên Cortex-M

- `.text` — mã lệnh, nằm trong Flash tại `0x08000000` — khái niệm giống AVR, chỉ khác địa chỉ cụ thể.
- `.data` — biến khởi tạo khác 0: giá trị ban đầu lưu Flash (LMA), sao chép sang SRAM (VMA) lúc khởi động bởi `Reset_Handler` — cơ chế **tường minh hơn AVR** (đã trình bày chi tiết ở tài liệu tham khảo, mục 8.2–8.3).
- `.bss` — biến không khởi tạo/=0 — giống AVR.
- **Heap** và **Stack** — cùng nguyên lý tăng dần/giảm dần đối đầu nhau, chỉ khác vùng SRAM lớn hơn (20KB thay vì 2KB) nên ”khoảng đệm an toàn” giữa hai vùng thường rộng rãi hơn.
- Khác biệt riêng của Cortex-M: có thêm `.isr_vector` — bảng vector ngắt đặt tại địa chỉ thấp nhất của Flash, mà AVR không có khái niệm tương đương ở dạng bảng địa chỉ tường minh này.

Sơ đồ bộ nhớ chương trình C trên Cortex-M



So với sơ đồ AVR (Tuần 3): thêm khối `.isr_vector` ở đầu Flash; nguyên lý Heap/Stack ”gặp nhau gây tràn” không đổi, chỉ khác biên độ an toàn nhờ SRAM 20KB thay vì 2KB.

Bài tập nhỏ: ước lượng chi phí biểu thức số học (Cortex-M)

Đề bài

Cho đoạn mã: `uint32_t a, b, c; float f; … c = a + b; f = f / 3.0;` trên STM32F103 (Cortex-M3, không FPU). Hãy sắp xếp 2 phép toán trên theo thứ tự **chi phí tăng dần** và giải thích.

- Gợi ý: so sánh 1 lệnh `ADDS` với việc gọi hàm thư viện chia `float` phần mềm.
- Đáp án được thảo luận trực tiếp trên lớp — so sánh thêm với đáp án tương ứng đã có ở Tuần 3 (AVR) để thấy rõ chênh lệch tuyệt đối giữa hai kiến trúc.

Kiểm tra nhanh — Câu hỏi

Thảo luận

1. Vì sao phép cộng `uint32_t` trên Cortex-M chỉ cần 1 lệnh máy, trong khi phép cộng tương tự trên AVR cần 4 lệnh?
2. Cortex-M3 có lệnh chia phần cứng, nhưng Cortex-M0 thì không. Điều này ảnh hưởng thế nào tới việc chọn chip cho một ứng dụng cần chia số nguyên thường xuyên?

Kiểm tra nhanh — Gợi ý đáp án

1. Vì ALU của Cortex-M rộng đúng 32-bit, khớp hoàn toàn với độ rộng `uint32_t` — phép cộng thực hiện trọn trong 1 lệnh `ADDS`. AVR có ALU chỉ 8-bit nên phải chia phép cộng 32-bit thành 4 bước byte-by-byte với lan truyền carry (1 `ADD` + 3 `ADC`).
2. Nếu chọn Cortex-M0 (giá rẻ hơn, tiêu thụ điện thấp hơn) cho ứng dụng chia nhiều, mỗi phép chia sẽ phải gọi hàm thư viện phần mềm (chậm, tốn Flash) — mất đi lợi thế phần cứng vốn có của họ Cortex-M nói chung; nên cân nhắc chọn Cortex-M3 trở lên (có `UDIV`/`SDIV`) nếu chia số nguyên là thao tác thường xuyên trong ứng dụng.

Thực hành — ARM Cortex-M: Buzzer & UART

Mục tiêu buổi thực hành (Cortex-M)

- Lập trình điều khiển loa Piezo (buzzer) trên board STM32 bằng kỹ thuật delay-based — cùng kỹ thuật đã học với AVR.
- Cấu hình và sử dụng giao tiếp USART cơ bản để gửi dữ liệu từ STM32 lên máy tính.
- Hoàn thành bài Lab: gửi chuỗi `"Hello Embedded System"` qua USART, quan sát trên phần mềm terminal (PuTTY/Termite).
- Kết hợp nhiều ngoại vi (nút nhấn, buzzer, USART) trong cùng một chương trình — cùng cấu trúc bài Lab như Tuần 3 AVR.

Chuẩn bị phần cứng (Cortex-M)

- Board STM32F103C8T6 (”Blue Pill”) hoặc Nucleo, mạch nạp ST-Link V2.
- Loa Piezo (buzzer) thụ động hoặc chủ động — cùng linh kiện như Lab AVR.
- 1 nút nhấn, breadboard, dây nối.
- Cáp USB kết nối ST-Link với máy tính; **cần thêm** bộ chuyển đổi USB-UART rời (vd CP2102, FTDI) hoặc dùng USART qua chân TX/RX nối trực tiếp — khác AVR (USB-Serial đã tích hợp sẵn trên board Arduino Uno).

Giới thiệu loa Piezo trên Cortex-M — không đổi

- Hiệu ứng áp điện (piezoelectric effect) — nguyên lý vật lý không đổi so với AVR.
- Buzzer chủ động và thụ động — phân loại và cách dùng không đổi.
- Bài Lab hôm nay vẫn dùng phương án thụ động — vi điều khiển (lần này là Cortex-M) tự tạo dao động bằng phần mềm, giống hệt cách tiếp cận ở AVR.

Sơ đồ đấu nối loa Piezo (STM32)



Chân PA1 điều khiển trực tiếp loa Piezo thụ động — cùng nguyên lý xung vuông đổi mức liên tục như chân D9 (PB1) trên AVR, chỉ khác chân vật lý và mức điện áp (3.3V thay vì 5V).

Nguyên lý tạo âm thanh bằng phần mềm — không đổi

- Vẫn cùng nguyên lý: đảo mức điện áp ở đúng tần số mong muốn (vd 1000 Hz $\rightarrow$ chu kỳ 1ms, mỗi nửa chu kỳ 500μs).
- Kỹ thuật delay-based: bật chân $\rightarrow$ trễ nửa chu kỳ $\rightarrow$ tắt chân $\rightarrow$ trễ nửa chu kỳ $\rightarrow$ lặp lại — không đổi.
- Khác biệt thực dụng: Cortex-M chạy ở 72MHz (so với 16MHz AVR) nên hàm trễ vi mô (`HAL_Delay` chỉ có độ phân giải mili-giây) **không đủ chính xác** cho việc tạo trễ nửa chu kỳ 500μs — cần dùng vòng lặp đếm hoặc Timer phần cứng thay vì `HAL_Delay` thô sơ.

Code C: phát âm thanh qua buzzer (STM32)

```c
#include "stm32f1xx_hal.h"
#define BUZZER_PIN GPIO_PIN_1

void delay_us(uint16_t us) {           // vong lap don gian, khong chinh xac tuyet doi
    for (uint32_t i = 0; i < us * 8; i++) { __NOP(); }
}
void phat_am_1khz(uint16_t so_chu_ky) {   // nua chu ky 500us ~ 1kHz
    for (uint16_t i = 0; i < so_chu_ky; i++) {
        HAL_GPIO_WritePin(GPIOA, BUZZER_PIN, GPIO_PIN_SET);   delay_us(500);
        HAL_GPIO_WritePin(GPIOA, BUZZER_PIN, GPIO_PIN_RESET); delay_us(500);
    }
}
int main(void) {
    HAL_Init();
    // ... khoi tao GPIOA PA1 output (nhu Tuan 2) ...
    while (1) {
        phat_am_1khz(1000);   // phat am ~1 giay
        HAL_Delay(500);       // nghi ~0.5 giay (HAL_Delay du chinh xac cho khoang nghi dai)
```

Nhắc lại giao tiếp UART/USART — khung truyền không đổi



- Khung truyền ”8N1” — hoàn toàn không đổi, đây là chuẩn giao tiếp vật lý độc lập với kiến trúc CPU.
- STM32 gọi ngoại vi này là **USART** (Universal Synchronous/Asynchronous Receiver/Transmitter) — có thêm khả năng đồng bộ mà AVR (UART thuần) không có, nhưng khi dùng ở chế độ bất đồng bộ thì hoàn toàn tương đương UART.

Cấu hình tốc độ baud trên STM32 — công thức khác AVR

- Tốc độ baud thiết lập qua thanh ghi `USART_BRR` — nhưng công thức tính **khác** công thức `UBRR` của AVR: STM32 dùng bộ chia tần số phân số (fractional baud rate generator).

$$\text{USARTDIV} = \frac{f_{PCLK}}{16 \times \text{baud}}$$

- Phần nguyên của USARTDIV nạp vào `BRR[15:4]` (Mantissa), phần thập phân nhân 16 rồi làm tròn nạp vào `BRR[3:0]` (Fraction) — phức tạp hơn công thức 1 bước của AVR, nhưng cho sai số nhỏ hơn ở nhiều tốc độ baud.
- Thực tế: STM32CubeMX/HAL tự tính giá trị này — lập trình viên hiếm khi phải tính tay như với `UBRR` của AVR.

Các thanh ghi USART chính trên STM32

Thanh ghi	Chức năng
`USART_BRR`	Thiết lập tốc độ baud (Mantissa + Fraction) — vai trò tương đương `UBRR0H/L` của AVR
`USART_SR`	Cờ trạng thái — bit `TXE` báo bộ đệm truyền rỗng (tương đương `UDRE0`)
`USART_CR1`	Bật/tắt thu-phát — bit `TE` (truyền), `RE` (nhận), `UE` (bật toàn bộ USART)
`USART_DR`	Thanh ghi dữ liệu — ghi để truyền, đọc để nhận (tương đương `UDR0`)

Cấu trúc thanh ghi tương ứng khá sát với AVR — chỉ tên gọi và một vài bit cấu hình bổ sung khác nhau.

Gửi ký tự qua USART bằng C (thanh ghi trực tiếp)

```c
#include "stm32f1xx.h"

void USART_transmit(unsigned char data) {
    while (!(USART1->SR & (1 << 7)));   // cho co TXE = 1 (bo dem rong)
    USART1->DR = data;
}
// Khoi tao USART1 (9600 baud, 8N1) thuong duoc HAL/CubeMX sinh san;
// cau hinh thanh ghi truc tiep phuc tap hon dang ke so voi AVR
// vi can bat clock ngoai vi (RCC) va cau hinh chan GPIO alternate function truoc.
```

Khác biệt thực dụng quan trọng: trên AVR chỉ cần ghi `UBRR0`/`UCSR0B`/`UCSR0C` là xong; trên STM32 còn phải bật xung clock cho khối USART (`RCC->APB2ENR`) và cấu hình chân GPIO ở chế độ Alternate Function trước khi USART hoạt động.

Lab: Gửi ”Hello Embedded System” qua USART — đề bài (STM32)

Yêu cầu

Viết chương trình STM32 khởi tạo USART1 (hoặc USART2) ở tốc độ 9600 bps (dùng HAL hoặc thanh ghi trực tiếp), gửi chuỗi `"Hello Embedded System"` lặp lại mỗi giây. Dùng phần mềm terminal (PuTTY/Termite/Serial Monitor của CubeIDE) ở 9600 baud để quan sát.

- Gợi ý dùng HAL: `HAL_UART_Transmit(&huart1, (uint8_t*)str, strlen(str), HAL_MAX_DELAY);` — ngắn gọn hơn viết hàm `UART_print()` thủ công.
- Nộp báo cáo: mã nguồn + ảnh chụp màn hình terminal — cùng định dạng báo cáo như Lab AVR.

Lab: Code mẫu gửi chuỗi qua USART (HAL)

```c
UART_HandleTypeDef huart1;   // duoc cau hinh boi CubeMX/HAL_UART_Init()

int main(void) {
    HAL_Init();
    // ... MX_USART1_UART_Init() do CubeMX sinh ...

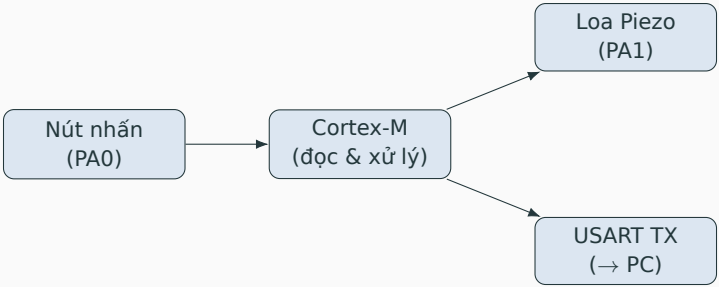
    char msg[] = "Hello Embedded System\r\n";
    while (1) {
        HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
        HAL_Delay(1000);
    }
}
```

So với bản AVR (viết tay `UART_print()` gọi từng ký tự): HAL cung cấp sẵn hàm gửi **cả chuỗi/mảng** trong 1 lời gọi — thể hiện đúng triết lý ”C + thư viện bậc cao giúp giảm code lặp lại” đã nêu ở mục 1.

Xem kết quả trên phần mềm terminal (STM32)

1. Nạp chương trình vào board qua STM32CubeIDE (nút nạp/debug, dùng ST-Link).
2. Kết nối bộ chuyển USB-UART rời (TX/RX/GND) vào đúng chân USART đã cấu hình — hoặc dùng cổng ảo tích hợp trên một số board Nucleo.
3. Mở phần mềm terminal (PuTTY, Termite, hoặc Serial Monitor tích hợp CubeIDE), chọn đúng cổng COM và tốc độ baud **9600**.
4. Quan sát dòng chữ `"Hello Embedded System"` lặp lại mỗi giây — cùng kết quả mong đợi như Lab AVR, khác công cụ quan sát (terminal rời thay vì Serial Monitor tích hợp Arduino IDE).

Mở rộng: Nhấn nút → phát âm thanh + gửi log USART



Cùng sơ đồ khối và cùng mục tiêu như bài mở rộng ở Tuần 3 (AVR) — chỉ đổi tên chân và tên ngoại vi truyền dữ liệu.

Code mẫu: kết hợp nút nhấn + buzzer + USART log

```c
#define BUTTON_PIN GPIO_PIN_0

int main(void) {
    HAL_Init();
    // ... khoi tao GPIOA PA0 input pull-up, PA1 output, USART1 ...

    while (1) {
        if (HAL_GPIO_ReadPin(GPIOA, BUTTON_PIN) == GPIO_PIN_RESET) {  // nhan = muc THAP
            phat_am_1khz(300);
            char msg[] = "Nut nhan duoc bam!\r\n";
            HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
            HAL_Delay(200);   // chong doi phim (debounce)
        }
    }
}
```

Kỹ thuật debug kiểu printf qua UART trên Cortex-M

- Cùng kỹ thuật debug đã học ở AVR: gửi giá trị biến qua UART/USART ra terminal thay vì dùng màn hình.
- Nâng cao hơn AVR: arm-none-eabi-gcc hỗ trợ **retarget** thẳng hàm `printf()` chuẩn C bằng cách override hàm `_write()` (semihosting hoặc qua USART) — STM32CubeIDE thường có mẫu sẵn cho việc này, gọn hơn cách ánh xạ `stdio.h` qua `fdevopen()` của avr-libc.
- Ưu điểm không đổi: không cần thiết bị debug chuyên dụng nếu chỉ cần xem giá trị biến — dù đã có ST-Link hỗ trợ debug thật (Live Expressions, đã học Tuần 2), debug qua UART/USART vẫn hữu ích khi cần quan sát log dài theo thời gian thực.

Bài tập về nhà (Cortex-M)

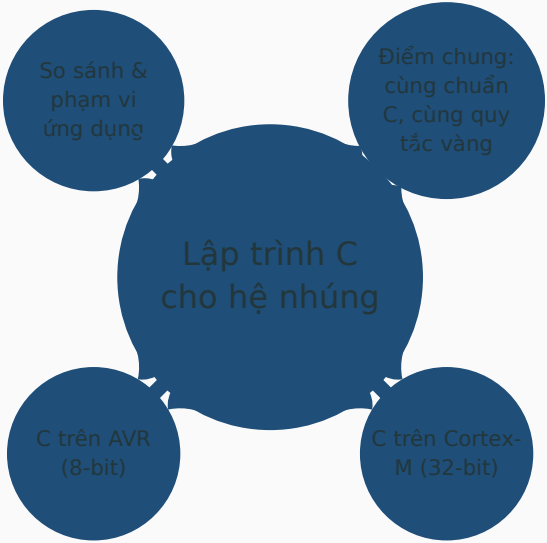
1. Thay đổi tần số phát của buzzer thành 500 Hz và 2000 Hz trên STM32 — so sánh độ chính xác tần số thực tế đo được (vd bằng oscilloscope hoặc app đo tần số) so với board AVR đã làm ở Tuần 3 trước.
2. Viết hàm gửi một số `uint32_t` thành chuỗi ký tự qua USART (không dùng `printf`), tương tự bài tập đã làm với AVR nhưng cho kiểu 32-bit.
3. Kết hợp nút nhấn để phát 2 âm khác nhau tương ứng 2 trạng thái, đồng thời log trạng thái qua USART — cùng yêu cầu như bài AVR.
4. So sánh dung lượng file `.bin` của chương trình Lab hôm nay với dung lượng `.hex` tương ứng ở Tuần 3 (AVR) — giải thích phần chênh lệch bằng kiến thức về `.isr_vector` và thư viện HAL đã học.

Tiêu chí đánh giá buổi thực hành (Cortex-M)

Tiêu chí	Điểm
Buzzer phát đúng âm thanh theo yêu cầu	2
Lab USART gửi đúng chuỗi ”Hello Embedded System”	4
Kết hợp nút nhấn + buzzer + USART log hoạt động đúng	2
Báo cáo rõ ràng, đầy đủ	1
Hoàn thành đúng thời gian	1

Tổng kết & So sánh: C trên AVR và C trên Cortex-M

Bản đồ tư duy — Lập trình C trên hai nền tảng



So sánh kiểu dữ liệu chuẩn C

Tiêu chí	AVR (avr-gcc)	Cortex-M (arm-none-eabi-gcc)
`int`	16-bit	**32-bit** (khớp PC)
`float` vs `double`	Giống nhau (đều 4 byte)	**Khác nhau** (4 byte vs 8 byte thật)
Con trỏ	16-bit (đủ cho $\leq$64KB)	32-bit
Kiểu khuyến nghị	`stdint.h` — bắt buộc thực tế	`stdint.h` — bắt buộc thực tế (giống hệt)

So sánh volatile / const / hằng số lớn

Tiêu chí	AVR	Cortex-M
`volatile` cho ISR	Bắt buộc	Bắt buộc (giống hệt)
`const` lưu ở đâu	SRAM (trừ khi dùng `PROGMEM`)	**Flash trực tiếp**, không cần chỉ thị đặc biệt
Đọc hằng số lớn	Cần `pgm_read_byte()`	Đọc thẳng như biến thường
Ánh xạ thanh ghi	Macro con trỏ từng thanh ghi	`struct` CMSIS ánh xạ cả khối ngoại vi

So sánh chi phí phép toán số học

Phép toán (32-bit)	AVR	Cortex-M3
Cộng/trừ	4 lệnh	1 lệnh
Nhân	Hàng chục lệnh	1 lệnh (`MULS`)
Chia	Luôn gọi hàm thư viện	1 lệnh `UDIV` (chỉ M3 trở lên)
`float`	Luôn thư viện phần mềm	Thư viện phần mềm (M3) / phần cứng nếu M4F+

So sánh bộ nhớ và giới hạn

Tiêu chí	AVR (ATmega328P)	Cortex-M (STM32F103C8)
SRAM	2 KB	20 KB
Flash chương trình	32 KB	64 KB
Bảng vector ngắt	Không có section riêng biệt	`.isr_vector` tường minh đầu Flash
Đệ quy/malloc	Tránh — SRAM rất hạn hẹp	Vẫn nên tránh, dù dư địa rộng hơn

So sánh công cụ và luồng biên dịch

Tiêu chí	AVR	Cortex-M
Trình biên dịch	avr-gcc	arm-none-eabi-gcc
Thư viện chuẩn	avr-libc	CMSIS + STM32 HAL
Linker script	Ẩn sẵn, hiếm khi chỉnh	Tường minh (`.ld`), CubeMX tự sinh
Đọc kết quả biên dịch	`avr-objdump`	`arm-none-eabi-objdump`

Điểm giống nhau cốt lõi: C vẫn là C

- Cú pháp ngôn ngữ (biến, hàm, `struct`, con trỏ, macro, `volatile`, `const`) **hoàn toàn không đổi** — vì đây là quy tắc của chuẩn C, không phải của phần cứng.
- Luồng biên dịch 4 bước (Preprocess $\rightarrow$ Compile $\rightarrow$ Assemble $\rightarrow$ Link) giống hệt về khái niệm, chỉ khác tên công cụ và chi tiết bước Link.
- Cấu trúc chương trình (khởi tạo 1 lần + super-loop, `main()` không return) không đổi.
- Các ”quy tắc vàng” của lập trình nhúng (tránh `float`/đệ quy sâu/`malloc` trong mã tần suất cao, luôn dùng `stdint.h`, luôn `volatile` cho biến chia sẻ ISR) **áp dụng như nhau** trên cả hai kiến trúc.

Phạm vi ứng dụng: C trên AVR khác C trên Cortex-M ở đâu?

- **Trên AVR**: lập trình viên phải ý thức rất cao về chi phí số học đa byte, tận dụng PROGMEM triệt để, tránh mọi phép toán trên kiểu $>$8-bit khi có thể — phù hợp dạy nhập môn vì buộc hiểu sâu ”cái giá thật sự” của mỗi dòng C.
- **Trên Cortex-M**: chi phí số học 32-bit gần như ”miễn phí”, SRAM dư dả hơn, const không cần xử lý đặc biệt — lập trình viên có thể tập trung nhiều hơn vào logic ứng dụng và ít phải vi tối ưu hoá thủ công.
- **Hệ quả thực tế**: các thuật toán vừa/nặng (xử lý tín hiệu số, giao thức phức tạp, RTOS đa tác vụ) hầu như luôn chọn Cortex-M; các ứng dụng cực đơn giản, giá cực rẻ, tiêu thụ điện cực thấp vẫn có chỗ đứng cho AVR.

Kết luận Tuần 3: Lập trình C trên AVR và Cortex-M

Tổng kết

Ngôn ngữ C mang lại đúng những lợi ích như nhau trên cả hai kiến trúc (năng suất, tính di động, an toàn hơn Assembly) — nhưng **”cái giá phải trả”** của từng thao tác cụ thể (số học đa byte, truy cập hằng số, thao tác bit GPIO) khác nhau đáng kể, phản ánh đúng sự khác biệt phần cứng đã học ở Tuần 2.

Hiểu rõ sự khác biệt này giúp sinh viên không chỉ ”viết C chạy được” trên một chip cụ thể, mà còn biết **vì sao** cùng một đoạn C lại có chi phí khác nhau khi đổi nền tảng — kỹ năng cốt lõi của một lập trình viên nhúng chuyên nghiệp.

Hết Tuần 3

Hẹn gặp lại ở Tuần 4: Bộ nhớ, Bus dữ liệu và Tối ưu hoá